

LLM-Assisted Trading Strategy Search: Validation and the Limits of Automated Alpha Discovery

Vincent Maciejewski
Independent Researcher
mayeski@gmail.com

October 2, 2026

Abstract

Large language models write trading-strategy code fluently, yet they cannot tell whether a strategy captures a real market effect or a pattern that only looks good in history. We study how to use them despite this. We present LLM-Assisted Trading Strategy Search (LATSS), an evolutionary program-search loop in the style of DeepMind’s FunSearch, in which the model proposes and edits the structure of a candidate strategy while deterministic components fit its numeric parameters and test the result against a skill bar, a statistical threshold for how much apparent skill a search of the given kind can produce by chance alone. Rather than claim a profitable strategy, we ask whether the loop can be trusted as a measurement tool, and we validate it on a controlled synthetic test-bed in which the amount of real signal is known and dialed from zero upward. Validating the loop as a measurement tool requires hundreds of runs, so the statistics are carried by cheap mechanical proposers, random and evolutionary program search run through the identical evaluation machinery, with the language-model arm run alongside them for comparison. The loop rejects noise and recovers genuine signal once it is strong enough; the test is sound only when the bar is calibrated on the same class of strategies the search can produce, and a bar calibrated on a smaller class lets pure noise pass; and on the test-bed’s deliberately meaningless features, where a semantic prior has nothing to use, the language model detects signal at the same threshold as mechanical search. We then explain why the approach does not invent new *alpha*, a genuinely new source of market edge. Alpha discovery is an inverse problem, inferring a model from noisy returns, and it admits many fits, so LATSS is its Bayesian-Popperian solution in the sense of Tarantola: a learned prior proposes models and a falsifying test keeps the survivors. A trained transformer is a fixed, stateless inference engine, and invention is not inference, since creating a hypothesis outside the model’s distribution is a different act from sampling within it. The model’s role is therefore to assist, as a fast generator of conjectures and an executor of search, while the step that originates new alpha belongs to the agent around the model. Our claim is that current systems do not take that step, and we propose a benchmark that would measure it. Code and experiments are open source.

Contents

1	Introduction	3
2	Prior art	4
3	The LATSS framework	5

4	The LATSS loop	5
4.1	Two ways to use the loop	5
4.2	Language model versus genetic search as the proposer	7
4.3	Construction and cost of the two generators	8
5	Validation methodology	10
5.1	The null-max skill bar	12
5.2	Null construction and its assumptions	14
5.3	Alternatives to the null-max bar	14
6	Experimental design	15
6.1	E3: estimation of the null-max bar	15
6.2	E1: false-positive rate and detection power	16
6.3	E2: search amplification	16
6.4	E5: bar misspecification	17
6.5	E4: search-budget dependence	18
6.6	E7: LLM versus mechanical search	18
7	Results	19
7.1	The null-max bar (E3)	19
7.2	Power curve and false positives (E1)	19
7.3	Search amplification (E2)	20
7.4	Same-class ablation (E5)	20
7.5	Search capacity (E4)	21
7.6	LLM and mechanical search (E7)	22
7.7	Implementation	23
7.8	Findings	23
8	The limits of automated alpha search	25
8.1	Search and abduction	25
8.2	FunSearch	25
8.3	Creating a search space	25
8.4	Evidence on machine invention	26
8.5	Financial alpha	27
8.6	Proposing hypotheses	27
8.7	Two ceilings	27
8.8	Levels of novelty	27
8.9	Current systems	27
8.10	Withholding the predictive feature (E12a)	28
8.11	The ceiling	29
9	Toward abduction of alpha	29
9.1	The structure of scientific discovery	29
9.2	The language model in the discovery cycle	29
9.3	Invention as an agent-level capability	31
9.4	From the validated search loop to an invention loop	31
9.5	Alpha discovery as an inverse problem	31
9.6	A benchmark for the jump	34
9.7	Strengthening the measurement tool	35

9.8	Enlarging the hypothesis space	36
9.9	Directions and milestones	36
10	Limitations	36
11	Conclusion	37

1 Introduction

An LLM can produce a valid, plausible-looking trading strategy on demand. What it cannot do is tell whether the strategy works. It has no reliable way to separate a real market effect from an artifact of the history it was fit to, and it has no cheap oracle to check its own output against, unlike a chess engine with an authoritative board or a theorem prover that verifies a proof. The output is fluent and confident with no built-in tie to the truth.

LATSS addresses this with a division of labor between the language model and a set of ordinary, non-learning software components. The language model supplies the proposals, and nothing else: it proposes and revises the structure of a candidate strategy. The non-learning side does the two things the model cannot do reliably: a numerical optimizer fits the strategy’s free parameters, and an evaluator decides whether a fitted candidate survives, using cross-validation and a skill bar. The design follows FunSearch [1], where an LLM proposes programs, a systematic evaluator scores them, and the best feed back into the next prompt. LATSS adapts the pattern to trading, whose scoring landscape is far noisier and easier to overfit than FunSearch’s exact-score problems.

The search is given a fixed set of input signals, which we call its *feature vocabulary*; these might be technical measures such as momentum or volatility, or indicators a researcher supplies. The search can build a strategy only by combining these signals, so the set of all strategies it could possibly express, its *strategy space*, is fixed by that vocabulary. The vocabulary therefore sets the outer limit of what the search can find. This paper separates two questions.

1. **Is the loop a reliable measurement tool?** Given a feature that genuinely predicts, does the search recover and certify it, and does certification reject noise? We answer this empirically (Sections 3–7) with synthetic controls of known signal strength. A central move is to use mechanical *searchers*, where by a searcher we mean any procedure that proposes candidate strategies for the loop to score (the language model is one; the mechanical ones propose by random or evolutionary program search instead). The mechanical searchers reuse the exact evaluation stack but use no model, so they cost no tokens, and we can replicate thirty to ten thousand times and treat the loop as an object of measurement. The certification machinery, a purged and embargoed cross-validation and a capacity-based null-max bar, is set out and compared with its alternatives in Section 5.
2. **Can the loop invent new alpha?** We argue it cannot, as a property of the approach rather than of any one implementation (Section 8). LATSS searches a supplied hypothesis space; it does not perform the abductive step that creates a new one.

An LLM-driven search can be turned into a trustworthy measurement tool for signal that a human proposes, which is the role our experiments support.

These results invite a sharper reading, developed in Section 9. In the philosophy of science discovery is conjecture and refutation: a guess is proposed and then kept or dropped against evidence. Alpha discovery is the inverse problem of inferring a model from noisy returns, which is ill-posed, and LATSS is its Bayesian-Popperian solution in the sense of Tarantola, a learned prior that proposes

models and a falsifying bar that keeps the survivors. A trained transformer is a fixed, stateless inference engine, and invention is not inference, since creating a hypothesis outside the model’s distribution is a different act from sampling within it. The model’s role is therefore to assist, as a fast generator of conjectures and an executor of search, while the step that originates new alpha belongs to the agent around the model. The paper’s claim is that current systems do not take that step, and we propose a benchmark that would measure it.

Contributions. (i) A controlled benchmark for LLM strategy-search systems, with synthetic features of known signal strength and mechanical searchers that reuse the same evaluation stack. (ii) A same-class empirical null-max certification procedure, and a demonstration that a bar computed over a class smaller than the search is invalid. (iii) A capacity experiment showing the certification threshold rises with search budget while the calibrated certification frequency on noise stays controlled. (iv) A vocabulary-boundary test showing that search does not recover signal excluded from its representation. We do not claim that LATSS discovers a profitable trading strategy.

2 Prior art

Program search with LLMs. FunSearch [1] established the propose–score–feedback loop with an LLM proposing and varying candidate programs, finding new cap-set constructions and improved bin-packing heuristics, and AlphaEvolve extends the pattern to a broad range of algorithm and system-design problems [22]. The common structure is an evolutionary search over programs in which the LLM supplies variation and a deterministic evaluator supplies selection.

Evolutionary trading-strategy systems. The closest prior art evolves trading strategies themselves with an LLM in the loop. MadEvolve applies FunSearch-style and AlphaEvolve-style evolution to quantitative-finance code, evolving feature sets, signal logic, and position sizing [23]; QuantEvolve couples MAP-Elites with island populations and hypothesis-driven mutation to evolve complete strategies as executable code [24]; and Alpha-GPT places a human in the loop, translating a researcher’s idea into mined alphas [25]. LATSS shares the propose-fit-evaluate loop but differs in emphasis. Its contribution is the certification discipline, a same-class null-max bar and the measurement of the loop as a tool, rather than a new evolutionary mechanism or a reported live edge.

LLM alpha mining. A growing family applies this loop to finance, searching formulaic factors over raw price and volume with a fixed set of mathematical operators: AlphaAgent [14], “Automate Strategy Finding with LLM” [15], CogAlpha [16], Hubble [17], QuantaAlpha [18], AlphaLogics [19], EFS [20], and reinforcement-learning miners such as AlphaGen [26], surveyed in [21]. These differ from LATSS in scope, searching factor formulas over raw inputs rather than strategies over given indicators, and in machinery, with multi-agent hierarchies, tree search, sandboxes, and novelty regularizers. They share one paradigm: recombination of a human-defined primitive vocabulary. Section 8 returns to what this line does and does not solve.

Overfitting and evaluation. Certification here rests on capacity-based reasoning about how much apparent skill luck can produce: VC-style bounds [12], empirical Rademacher complexity [13], the validity of cross-validation for time-series prediction [27], purged and embargoed cross-validation [9], the Deflated Sharpe Ratio and the probability of backtest overfitting [10, 28], and the null-max

construction [11]. We use an empirical null-max bar as the operative gate for the reasons in Section 5.1.

3 The LATSS framework

LATSS searches for a trading strategy expressed as a function of the features, implemented literally as a Python function the language model writes, the object a quantitative analyst otherwise hand-designs. The architecture is not tied to a particular market.

Problem setup. A human first sets up the problem by hand, and these choices are the inputs to LATSS: the universe of assets to trade, the set of input features a strategy may use (its *feature vocabulary*), and the benchmark the strategy is scored against. Together they define the *search space*, the set of all strategies that can be written as a function of the chosen features. This is the space the loop explores, and it is also its outer limit: a strategy that cannot be expressed from the chosen features is unreachable no matter how long the search runs. What LATSS then automates is the work the analyst would otherwise do inside that space, writing candidate strategies, fitting and scoring them, and keeping the best.

4 The LATSS loop

The loop has four parts.

1. **Representation.** Each candidate strategy is written as a small Python function, a *scoring function* `score(features, p)`, that reads the features for one asset and returns a number, its score, using a few numeric parameters p such as lookback lengths or weights. The strategy acts on the scores, for example by holding, each period, the highest-scoring asset. Writing a strategy this way is what lets the language model author and edit it as code, and it is the object the rest of the loop fits and tests.
2. **Search.** A language model proposes and revises the scoring function; this is its only role. It never sees prices and never picks a numeric value; it sees the scored history of previously proposed structures and emits a new structure. Differential evolution then fits p .
3. **Fitness.** Each candidate is backtested and scored by its Sharpe ratio (its average return divided by the volatility of that return, a standard risk-adjusted score), cross-validated out of sample and measured against a benchmark, the passive alternative for the use case. For a universe selector the benchmark is the equal-weight portfolio, so the score is the *active Sharpe*, the Sharpe of the strategy's return minus the benchmark's, which credits only the margin over the benchmark. The loop keeps the highest-scoring candidate, the champion.
4. **Certification.** The champion is credited with skill only if its active Sharpe clears a skill bar (Section 5.1).

4.1 Two ways to use the loop

There are two ways to use LATSS. The first is *generative*: you have a set of candidate indicators you believe might predict returns, and you run the loop to find a strategy that combines them. The

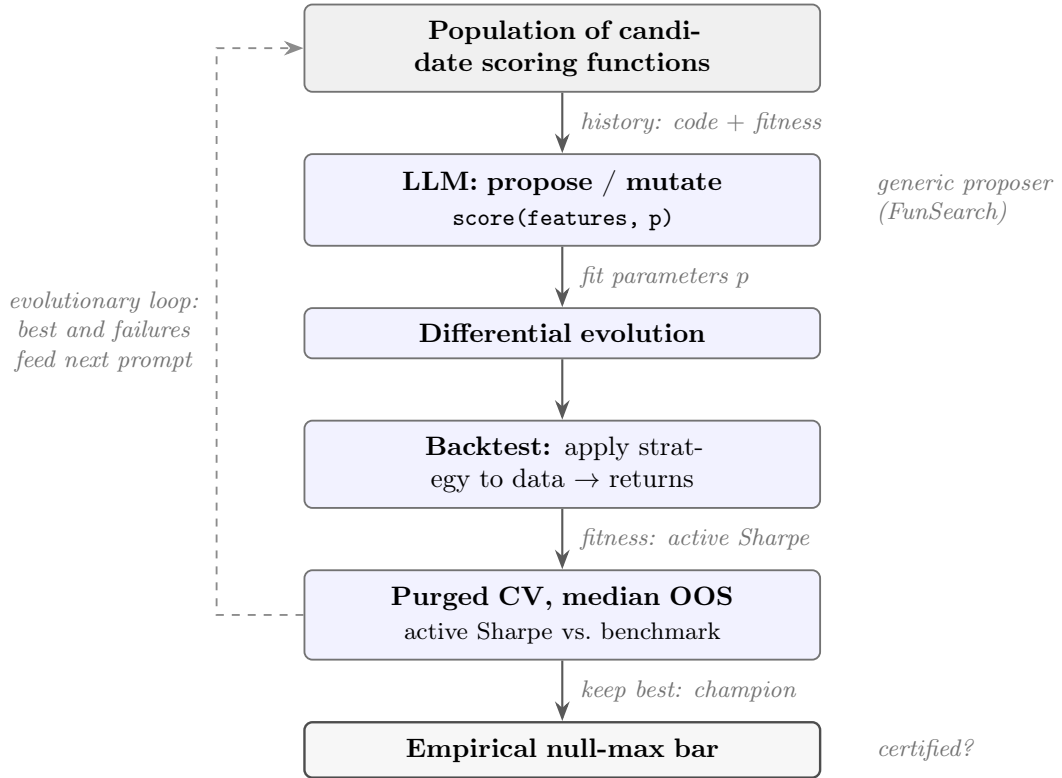


Figure 1: The LATSS loop. The LLM proposes and mutates strategy structure from the scored history, differential evolution fits parameters, and the evaluator scores by cross-validated active Sharpe and certifies the champion against the null-max bar.

search will overfit if left unchecked, so you trust the result only if it clears the skill bar, the statistical test a genuine edge must pass.

The second is *diagnostic*: you are developing a single indicator and want to know whether it actually carries signal. You give it to the loop mixed in with several pure-noise indicators, without telling the loop which is which. If the loop repeatedly singles out your indicator and its strategy clears the bar, run after run, that is evidence the indicator is real rather than a lucky fit.

Both uses rely on the same thing: the loop must reject noise and recover a genuine signal once it is strong enough. That is the property the rest of the paper tests.

4.2 Language model versus genetic search as the proposer

The proposal step of the loop does not require a language model. A classical genetic or random search fills the same role: fix a grammar of expressions over the features, meaning a list of allowed operations such as addition, multiplication, and a few transforms, together with a maximum formula size, and generate or mutate formulas mechanically. Section 5 builds exactly such searchers; they cost milliseconds per candidate instead of a model call, and they carry most of the statistics in this paper. The language model therefore has to justify its place in the loop. Two properties argue for it, and one open question decides whether it earns the position; Table 1 summarizes the reasons and what this paper can say about each.

Table 1: Reasons for using a language model as the proposer, and the status of each in this paper.

Reason	What it buys	Status
Open representation	Any short Python function is reachable; no grammar must be designed, and forms the designer did not anticipate cost nothing	Structural property; demonstrated for program search in general by FunSearch [1], not separately measured here
Semantic prior	On features with meaningful names, proposals cluster around economically plausible combinations, so fewer evaluations may be needed	Untested here: the test-bed’s features are opaque by design, so the prior is inert; the deciding experiment is future work (Section 9.7)
Readable candidates	Each champion is commented code a practitioner can audit, not an expression tree	Holds by construction
Steering in English	The search is redirected by editing the prompt, not by rewriting a generator	Holds by construction
Detection power on opaque features	None expected, since the prior has nothing to read	Measured: parity with mechanical search at every signal strength (E7, Section 7.6)

The first property is that the model removes the need to design the search space by hand. A genetic search can only emit what its grammar generates, and the grammar is a design decision made before the search starts: which operations are allowed, which transforms exist, how large a formula may grow. Whatever the designer leaves out is unreachable no matter how long the search runs. The model proposes arbitrary short Python functions, so the space automatically includes conditionals, intermediate variables, and combinations of operations that no grammar was written to produce. This is the argument from program search generally: FunSearch [1] paired a language

model with an evaluator precisely because mutation in an open programming language reached programs that hand-built mutation operators did not.

The second property is that the model’s proposal distribution is a prior rather than a blind draw. A mechanical searcher samples its grammar uniformly; every well-formed formula of a given size is as likely as any other, including the economically absurd ones. The model samples from its training distribution, which concentrates probability on code resembling what people have written about the problem. When the features carry meaningful names, such as momentum, book-to-price, or realized volatility, the model’s proposals cluster around combinations that published economic reasoning considers plausible, so the search may reach a good candidate in far fewer evaluations than blind search needs. This prior is the only channel through which a language model can add detection power over a mechanical search of the same vocabulary; representation aside, everything else the model does, a mechanical searcher does for free.

The measured results of this paper bound the second property rather than confirm it. The synthetic test-bed strips the feature names deliberately, because controlling the signal strength requires features with no meaning, so the prior has nothing to act on there. E7 (Section 7.6) measures the consequence: with opaque features, the language model detected signal at the same threshold as mechanical search and showed no advantage. The prior also has a known cost in this domain. The ideas most strongly represented in the training distribution are the published ones, and in trading a published signal is a crowded one, which is why LLM alpha-mining systems add explicit penalties against regenerating known factors [14]; Section 8 develops this point.

The position of this paper is therefore the following. The certification machinery of Section 5 is the substance of the method and works identically for any proposer. The language model is one proposer among several, kept for the open representation and for readability, since each champion is commented code a practitioner can audit rather than an expression tree. Whether its semantic prior buys detection power on a vocabulary of nameable features is the open question: it is not measured here, and we are aware of no controlled measurement of it elsewhere. The power-curve methodology of this paper applied to a nameable vocabulary, stated as future work in Section 9.7, is the experiment that would decide it. If the prior buys nothing there, then the language model is a convenience of representation and the loop is a genetic search with a well-calibrated test.

That outcome would not contradict the thesis of this paper; it is what the thesis predicts. The central claim of Sections 8 and 9 is that the model does not originate alpha: it proposes candidates from inside its learned distribution, and the value it adds over blind proposal can only come from that distribution being a useful prior. On a vocabulary with no semantics the prior is useless by construction, and the measured parity of E7 is the expected result. The contribution of the paper does not rest on the model outperforming a genetic search. It rests on the certification machinery, which is proposer-independent, and on the ceiling argument, for which E7’s parity is supporting evidence rather than a failure.

4.3 Construction and cost of the two generators

The previous subsection gave the arguments; this one compares the two generators as working machinery, because the difference is concrete. Every search needs a component that produces the next candidate scoring function, and the two paths build that component in entirely different ways.

Generation with the language model. The generator is a prompt. The loop assembles the scored history, the source code of previously tried scoring functions together with their cross-validated scores, strongest first, and sends it to the model with a fixed task description. The model returns the Python source of one new scoring function. No part of the space of possible strategies is written

down anywhere: the model may emit any function of the features that is valid Python, including conditionals, intermediate quantities, and compositions nobody specified. The engineering burden sits elsewhere. The emitted code is untrusted text, so the loop must parse it, reject code that does not compile or that references anything outside the allowed inputs, execute it in a restricted environment, and handle the fraction of proposals that fail these checks. Steering the generator is done in English: adding a feature means naming it in the prompt, and biasing the search toward or away from a style of rule means editing a sentence.

Generation without the language model. The generator must be coded by hand, and coding it means writing down, in advance, every form a candidate may take. For the nonlinear searcher of this paper that meant choosing the full inventory explicitly: the four features as leaves; five unary operations (negation, absolute value, sign, clipping, cross-sectional standardization); seven binary operations (addition, subtraction, multiplication, a division protected against zero denominators, maximum, minimum, and a threshold gate); a maximum expression depth of three; and numeric constants drawn at random and fixed into the expression. On top of the inventory sit two routines: one that samples a random expression tree from these pieces, and one that compiles a tree into an executable scoring function. The whole generator is a few hundred lines, written once. Its candidates are valid by construction, so no parsing or sandboxing is needed, and each one costs about four milliseconds to generate and evaluate. The linear searcher is simpler still: the form is fixed as one weight per feature, and generation is nothing more than letting the optimizer move the four weights.

The comparison. Each path is better at what the other is worst at.

- *Coverage of the space.* The hand-coded grammar reaches exactly what its inventory implies and nothing else; any operation the designer did not anticipate is unreachable at any budget. The model reaches any short program, so forms outside the designer’s imagination cost nothing. This is the language model’s genuine structural advantage in generation.
- *Cost.* A mechanical candidate costs milliseconds and no tokens; a model-generated candidate costs a network call, seconds of latency, and metered tokens, a difference of several orders of magnitude per candidate. This is why the mechanical searchers carry the statistics of this paper, 30 to 10,000 candidates per run across hundreds of runs, while the language-model arm has nine runs.
- *Cost of the bar.* Certification requires rerunning the full search on roughly one hundred scrambled datasets (Section 5.1), so the bar costs one hundred searches, whatever one search costs. For the mechanical searcher that is minutes; for the model-driven searcher it is one hundred times the token cost of the original run. The open representation that is the model’s advantage in coverage directly inflates the price of certifying what it finds.
- *Validity and audit of the class.* The grammar states the strategy class exactly, which makes the same-class condition easy to satisfy and easy to verify. The model’s class is implicit, namely whatever it tends to emit under the prompt, so the only way to characterize it is empirically, by running it.
- *Quality of the draw.* The grammar samples blindly; the model samples from a prior (Section 4.2). On nameless features the prior is inert and E7 found parity. On nameable features the prior is the model’s one potential advantage in proposal quality, and it is unmeasured.

On every axis measured in this paper, cost, statistics, and ease of certification, the hand-coded generator wins, and nothing in our results justifies the model’s price on an opaque vocabulary. The axes on which the model wins, coverage without design and a prior that can read feature names, are respectively unpriced and unmeasured here, and they are the reason the model stays in the loop as the default proposer for real feature sets, where the vocabulary is nameable and the space of sensible rules is not known in advance.

5 Validation methodology

A search cannot be validated by the scores it assigns itself, so we test the loop the way one would test a new measuring device: on problems where the right answer is known in advance. We build a small laboratory version of the market in which we control exactly how much real signal exists, from none at all to plenty, by planting it ourselves. We then run the loop many times over and ask three questions. When there is nothing to find, does it correctly find nothing, or does it hand us a strategy anyway? When there is something to find, does it find it? And how strong does the signal have to be before it does?

Answering these takes two pieces of machinery beyond the loop itself. The first is a way of scoring candidates that cannot leak future information into the past, which is the cross-validation. The second is a way of knowing how good a score luck alone can produce, which is the skill bar, and it is needed because a search that tries thousands of candidates will always find something that looks good; the question is never whether the best candidate looks good but whether it looks better than the best accident. Everything in this section is one of those pieces: the market we test on, the planted signal, the scoring, the searchers that let us repeat the experiment at scale, and the bar.

Test-bed. The universe is the seven “Magnificent-7” US equities (AAPL, MSFT, GOOGL, AMZN, NVDA, META, TSLA), daily from 2015 to 2026 (2,952 bars, about 11.7 years). The universe is fixed in advance and the predictive features are synthetic, so survivorship and universe-selection effects do not enter these experiments. Each bar the strategy holds the single highest-scoring name, and the benchmark is the equal-weight portfolio. The metric throughout is the *active Sharpe*. Let r_t^{strat} be the strategy’s net daily return (the return of the name it holds on day t , after costs) and r_t^{bench} the net daily return of the equal-weight portfolio of the seven names. The active return is their difference, $a_t = r_t^{\text{strat}} - r_t^{\text{bench}}$, and the active Sharpe is its annualized mean-over-standard-deviation,

$$\text{active Sharpe} = \frac{\bar{a}}{\text{sd}(a)} \sqrt{P}, \quad P = 252 \text{ trading days per year}, \quad (1)$$

where $\bar{a}$ and $\text{sd}(a)$ are the sample mean and standard deviation of the daily active returns. Subtracting the benchmark isolates selection skill: holding the benchmark gives $a_t = 0$ and an active Sharpe of zero. A single name inherits the basket’s bull-market trend, so its standalone Sharpe would credit that trend rather than the skill of picking it.

Synthetic features. A feature here is an input signal, one number per name per day, that the scoring function reads; it is the raw material a strategy is built from, in the way a momentum or volatility measure would be. It is not itself a strategy or a function. Instead of real indicators we supply synthetic signals whose predictive content we set exactly, so signal and noise are known ground truth. Each name is given four such signals, `feat_a` through `feat_d`. In the negative control all four are persistent, unit-variance noise: each day’s value is 0.9 times the previous day’s plus a fresh random shock (a first-order autoregressive process, AR(1), with coefficient $\phi = 0.9$), so the

series drifts slowly and looks like a plausible indicator rather than day-to-day random noise, which makes it a harder decoy for the bar to reject. In the positive control one hidden slot (**feat_c**) carries a cross-sectional predictor: for every name n and day t ,

$$\text{feat_c}[n, t] = \rho z[n, t] + \sqrt{1 - \rho^2} \varepsilon[n, t], \quad (2)$$

where $z[n, t]$ is the signal, a standardized view of the name’s future return, and ε is independent unit-variance noise. The intuition is a single dial $\rho \in [0, 1]$ that mixes signal and noise. Because z and ε are both unit-variance and independent, the weights ρ and $\sqrt{1 - \rho^2}$ are exactly the ones that keep the total variance equal to one for every ρ ($\rho^2 + (1 - \rho^2) = 1$). This is not vol normalization; it means turning the dial changes only how much of the feature is signal, not its scale. At $\rho = 0$ the feature is pure noise, at $\rho = 1$ it is the pure predictor, ρ^2 is the fraction of its variance that is signal (equivalently ρ is its correlation with z), and the constant scale matters because it stops the searcher from detecting signal from the feature’s magnitude: only its relationship to returns changes (Figure 2). Concretely, the signal z is built in two steps. First, $\text{ex}[n, t]$ (short for excess return) is how much name n beats the universe over the next $h = 21$ trading days: its raw forward return $p[n, t+h]/p[n, t] - 1$ minus the average forward return across the seven names on day t . Second, $z[n, t]$ standardizes this per name, subtracting ex ’s mean and dividing by its standard deviation (computed over the full sample), so z is unit-variance. The searcher is told none of this. **feat_c** is a deliberate synthetic oracle built from future returns, used to set a known signal strength; it is the predictor under test rather than a tradable feature, and is the only route by which forward information enters the experiment. One subtlety: the standardization of z divides by the mean and standard deviation of ex computed over the whole sample, which uses data a live strategy could not see. In a real predictor that would be a look-ahead error. It is harmless here because z is not something anyone trades; it is the synthetic answer key we inject, and the whole-sample standardization only calibrates how strong the injected signal is. It changes nothing about how a strategy is backtested.

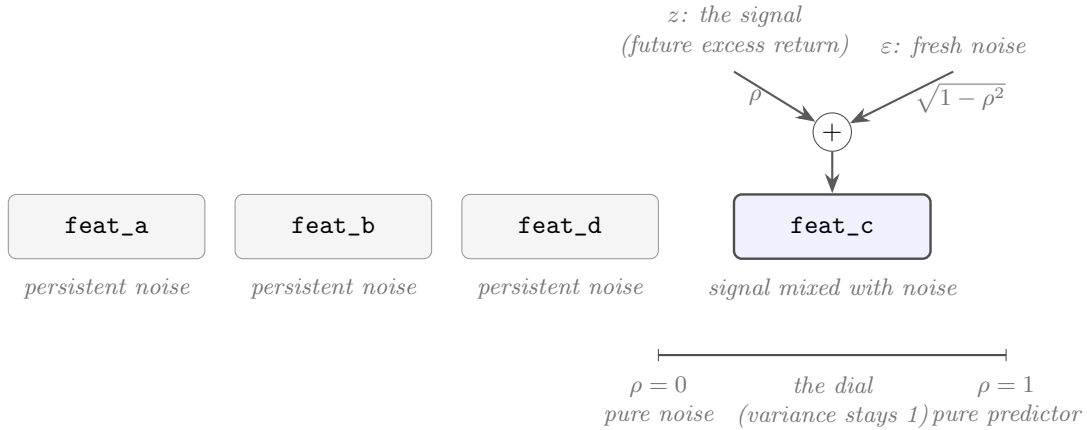


Figure 2: The four input features of the test-bed. Three are persistent noise. The fourth, **feat_c**, mixes a known signal z with fresh noise through the dial ρ of Eq. (2): at $\rho = 0$ it is pure noise, at $\rho = 1$ the pure predictor, and its scale is the same at every setting, so only its signal content changes.

Cross-validation. We use purged, embargoed, chronological k -fold [9]. The pool is cut into six contiguous blocks (Figure 3); each serves once as the test fold, and training is every other bar except an embargo band of 250 trading days on each side of the test block. The embargo exceeds the

longest feature lookback, so no training bar shares a lookback window with a test bar. We avoid the common alternative of random 75/25 quarter splits, in which adjacent quarters overlap through warm-up and the split is not chronological. A permissive scheme is doubly damaging, because fitness and the noise bar are computed under the same cross-validation, so leakage inflates a signal’s score and lowers the bar at the same time.

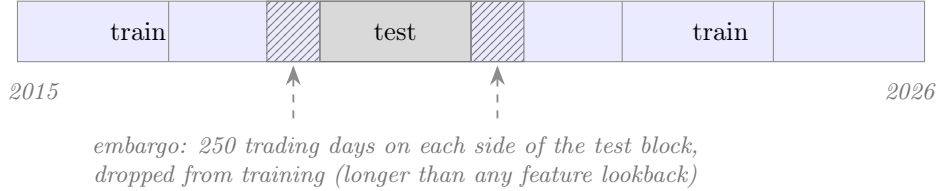


Figure 3: The purged, embargoed cross-validation. The eleven years are cut into six blocks and each block takes one turn as the test set, with the rest used for training except an embargo band on each side of the test block, dropped so that no lookback window spans both training and testing. The figure shows one of the six folds.

Searchers. The LLM proposes structures as described in Section 7.6. Measuring the loop statistically means running it hundreds to thousands of times, which is too slow and costly with LLM calls, so in place of the model we also use three mechanical searchers that plug into the same scoring contract and evaluation stack. Each one proposes scoring rules from a different family (a strategy *class*, the term used below), and the families differ in how flexible a rule they allow. The *linear* searcher proposes only weighted sums of the four features, $w_a \text{feat_a} + w_b \text{feat_b} + w_c \text{feat_c} + w_d \text{feat_d}$, with the four weights fit by the numerical optimizer; these are the simplest rules available. The *nonlinear* searcher generates random formulas that combine the features more freely, multiplying them, clipping them, or switching one on and off depending on another; each formula’s constants are set when it is generated, so no fitting is needed and trying ten thousand candidates is cheap. The third evolves such formulas by mutation and selection, ordinary genetic programming. The nonlinear family matters most for the skill bar: because it can express far more shapes of rule, it is also far better at fitting pure noise, so it is the hardest test of whether the bar holds. It stands in for an LLM writing arbitrary code, but only as a stand-in, since its formulas are drawn from a fixed menu of operations while arbitrary code has no such limit (Eq. (3) returns to this).

Search capacity, evaluation capacity, and search budget. Three different things are easy to conflate here, so we give each a name. *Search capacity* is how flexible the family of rules is, how many different shapes of rule it can express at all; the linear family above is small in this sense, the nonlinear family large. *Evaluation capacity* is how much tuning happens inside each single candidate when the optimizer fits its parameters to the data. *Search budget* is simply how many candidates the search tries. All three are routes to fitting noise, and they need not move together. The nonlinear mechanical searcher has a fixed search capacity and a dial for its budget, while an LLM writing arbitrary code has an effectively unlimited search capacity. The experiment in Section 7.5 turns the budget dial alone and holds everything else still.

5.1 The null-max skill bar

Under a sufficiently expressive, persistent search no partition of the data is truly out of sample (Section 8), so we do not certify against a hold-out. We certify in sample against a null-max bar [11], the highest cross-validated active Sharpe the strategy class reaches on data whose signal has been

destroyed. A champion that cannot beat what its own class extracts from noise has no measured skill.

A VC-inspired capacity scale. The VC dimension of a family of rules, written h , is a standard measure of its flexibility [12]: roughly, the largest number of data points the family can fit in every possible configuration. A more flexible family has a larger h , and with it a greater ability to manufacture apparent skill from noise. For a family of VC dimension h evaluated over m trading sessions spanning Y years, luck alone cannot exceed an annualized Sharpe of order

$$\text{bar}_{\text{VC}} \approx \sqrt{\frac{2h \log(em/h)}{Y}}. \quad (3)$$

This is heuristic dimensional analysis, a VC-inspired capacity scale rather than a bound on Sharpe derived for our exact procedure: the constant does not fold in the argmax selection step or the dependence across observations. As an order of magnitude, a linear combination of the four features ($h \approx 5$) sits near 2.5 at our sample size, while an unconstrained code-writing class has effectively unbounded h and no finite scale.

Empirical bar (null-max, operative). The VC scale is adversarial, a ceiling rather than a working hurdle. The operative bar is measured on the data itself, using what we call a *scramble*: a copy of the return data in which the signal has been deliberately destroyed but everything else is left in place. One scramble is produced and used as follows.

1. For every name n and every day t , flip an independent fair coin $\sigma[n, t] \in \{-1, +1\}$ and set the scrambled return to $\tilde{r}[n, t] = \sigma[n, t] r[n, t]$. Each daily return keeps its magnitude and randomly keeps or reverses its direction, so any relation between the features and the returns is destroyed while the size and texture of the returns are preserved. (The sign flip is a Rademacher-style randomization.)
2. Recompute the equal-weight benchmark from the scrambled returns.
3. Run the identical search on the scrambled data: same family of rules, same search budget, same parameter fitting, same cross-validation, features untouched.
4. Record the best cross-validated active Sharpe that search finds, the most skill it could fake on data containing none.

Repeating this for many independent scrambles, one hundred to one thousand in our experiments, gives a distribution of best-on-noise scores. The bar is its 95th percentile, written bar_q (Figure 4). The VC expression is a conservative capacity-based reference scale; the empirical null-max bar is the operative calibration.

Same-class condition and certification. What must match is the search procedure itself. The bar is built by running a search on scrambled data, and the champion is produced by running a search on the real data; the two searches must be the same procedure in every respect, with the data as the only difference. The same respects are: the family of rules the search may emit (linear weights versus unrestricted expression trees), the search budget (how many candidates are tried), the parameter fitting applied to each candidate, and the cross-validated score used to rank them. We call a bar *matched* when all of these agree with the search that produced the champion, and *mismatched* when any of them differs in a way that changes how well the search fits noise. A matched

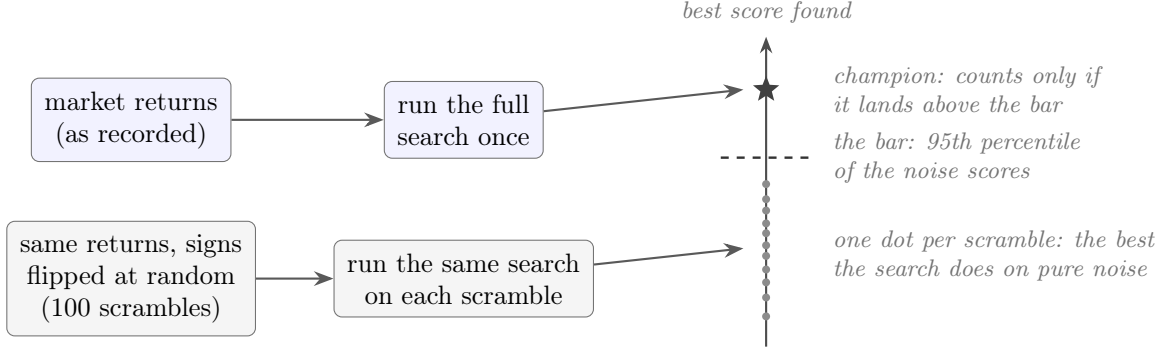


Figure 4: How the null-max bar is built. The same search runs once on the real returns, giving the champion, and once on each of one hundred copies whose signs are flipped at random, which removes any relation between features and returns. Each scrambled run records the best score the search can squeeze out of pure noise; the 95th percentile of those scores is the bar, and the champion is credited with skill only if it beats it.

bar answers the right question, namely what score this exact search achieves when there is nothing to find. A mismatched bar answers that question for some other search: computed over a smaller class, or with a smaller budget, it underestimates the achievable noise fit and admits overfits, which Section 7.4 measures. A consequence of the condition is that the bar is not reusable. Any change to the search, a new feature, a larger rule family, a higher budget, or a different fitting procedure, invalidates the previously computed bar, and a new bar must be computed by running the modified search on the scrambled data again. This recomputation is the recurring cost of the method. When the cost is paid the guarantee holds: Section 7.5 measures a false-positive count of 0 of 30 at every budget from ten to ten thousand candidates, each budget judged against its own recomputed bar. A champion is null-certified (it passes the null-max test) when its cross-validated active Sharpe exceeds bar_q ; we also report the margin $\eta = \text{CV}^2 / \text{bar}_q^2$, where $\eta \geq 2.5$ marks a comfortable clear. The VC scale is reported as a ceiling and is never used as the gate. Null certification means only that the observed champion exceeds the empirical 95th-percentile null maximum under the stated class, search procedure, and null construction. It is not a guarantee of future profitability.

5.2 Null construction and its assumptions

The sign flip multiplies each entry of the return matrix by an independent ± 1 . This removes the feature-to-return relation, and with it the temporal dependence, the cross-sectional and common-factor structure, and the volatility-state relationships present in the real panel. The bar is therefore the best in-class fit under a null in which returns are independently sign-randomized conditional on their magnitudes. It is a calibrated null reference for this selector under a directional-exchangeability assumption: absent a feature-to-return relation, the sign of each name’s return is exchangeable at the resolution the selector acts on, and the long-only top-one selection earns the held name’s return, whose sign the flip removes. The bar’s operating characteristics under richer dependence structures are not established here. Nulls that preserve more structure, a by-day sign flip, a cross-sectional permutation of returns across names, and block or stationary bootstraps, would test robustness to that assumption, and we leave them to future work.

5.3 Alternatives to the null-max bar

The standard alternatives are each unsuited to this setting. The Deflated Sharpe Ratio [10] deflates by a trial count, but for an automated search the trial count is ill-defined, since the search effectively

covers the whole class, and it is gameable because fewer trials incur a smaller penalty. The invariant that matters is the class’s capacity, which the null-max bar measures. Permutation and shift-the-signal tests select by a uniform maximum over sampled configurations, while the fitness selects by differential evolution, which digs deeper into noise, so they under-price the overfitting that produced the champion; we keep shift-the-signal only as a reported diagnostic. A hold-out is not automatically sufficient when it becomes part of an iterative research loop: with an expressive class and repeated adaptive re-use of the reserved period across attempts, the hold-out is fit too. A genuinely untouched hold-out, used once at the end, does give protection; the concern here is the iterative case.

6 Experimental design

Each experiment answers one of the laboratory questions of Section 5. This section specifies the experiments; Section 7 reports the results. Throughout, the injected predictor `feat_c` literally contains future returns (Eq. (2)): it is a calibrated oracle used solely to place a known amount of signal on a controlled scale, not a tradable or economically causal feature and not a simulated look-ahead backtest, and no part of the search or the bar sees its construction. The experiments measure signal recovery, not a trading result. All runs use purged, embargoed cross-validation and the empirical null-max bar; “clears” means the champion’s cross-validated active Sharpe exceeds the bar; mechanical conditions use at least 30 independent seeds and the LLM condition 3 seeds per point. The scripts producing every table are in the repository.

6.1 E3: estimation of the null-max bar

We need the numerical value of the bar before any other test can be read. To obtain it, we take the real returns and remove their signal with the sign-flip scramble of Section 5.1: every daily return keeps its size, and its direction is set by an independent coin flip, so no feature can predict the result. We then run the identical strategy search on the scrambled data and record the best cross-validated score it finds. Any score earned on this data is overfitting by definition, because there is nothing real to find. We repeat this up to one thousand times, each time with a fresh scramble, and collect the best scores into a distribution. The bar is the 95th percentile of that distribution. The experiment also measures how many scrambles are needed before that percentile settles to a stable value, which fixes how many repetitions the later experiments use.

Inputs. The Magnificent-7 daily panel of Section 5 (2,952 trading days), the all-noise feature set (four persistent noise series per name, no planted signal), and the two searcher families of Section 5: the nonlinear family of random formulas and the linear family of weighted sums.

Procedure. For each of 1,000 repetitions: (1) draw an independent ± 1 coin flip for every name and every day and multiply each daily return by it; (2) recompute the equal-weight benchmark from the flipped returns; (3) run the searcher on the flipped data exactly as it would run on real data, the nonlinear searcher drawing 200 random formulas of depth at most three and the linear searcher fitting its four weights by differential evolution with a budget of 150 evaluations; (4) score every candidate by purged cross-validated active Sharpe and record the single best score of the repetition. The procedure is run separately for the two families, because each must be measured against its own behavior on noise.

Outputs and analysis. The 1,000 recorded best-on-noise scores form the null distribution of each family. We report its mean and its 95th percentile, which is the bar. To determine how many repetitions the estimate needs, we also recompute the 95th percentile from the first 30, 50, 100, 250, and 500 repetitions and compare: the count at which the value stops moving is the repetition count the later experiments must meet. A family whose bar is still drifting after 1,000 repetitions would mean the later certifications rest on an unstable threshold, which is why this experiment runs first.

6.2 E1: false-positive rate and detection power

This is the main test of the loop. We build the test data so that the right answer is known in advance: the feature `feat_c` is constructed from the future returns themselves, so it genuinely predicts, and the parameter ρ of Eq. (2) sets how strongly, from not at all at $\rho = 0$ to almost perfectly at $\rho = 1$. At each value of ρ we run the complete search thirty times with different random seeds and count how often the champion’s score exceeds the bar. At $\rho = 0$ there is nothing to find, so every run that exceeds the bar is a false positive, and the count measures the false-positive rate. At higher ρ the count measures detection: how often the loop finds the planted feature and passes the bar when a real signal is present. Plotting that frequency against ρ gives the power curve of the loop. The value of ρ at which the curve rises from near zero to near one is the weakest signal the loop can reliably certify.

Inputs. The same panel; feature sets generated at nine signal strengths, $\rho = 0, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40, 0.50$, and 0.70 , each with `feat_c` built from Eq. (2) and the other three features noise; and the two bars measured in E3, $+0.844$ for the nonlinear family and $+0.408$ for the linear family.

Procedure. For every combination of signal strength and searcher family we run the complete search 30 times. The runs differ in the searcher’s random seed, which changes the formulas the nonlinear searcher draws and the optimizer’s path for the linear searcher; the feature realization is held fixed within each signal strength so the runs face the same data. Each run returns one champion, the candidate with the highest purged cross-validated active Sharpe, and that score is recorded. A run counts as a certification when its champion’s score exceeds the bar of its own family.

Outputs and analysis. For each combination we report the certification frequency out of 30 and the median champion score. The frequency at $\rho = 0$ is the observed false-positive rate; with 30 runs and zero observed false positives, the true rate is below 11.6% at 95% confidence (Clopper–Pearson). The frequencies across ρ form the power curve. From it we read the threshold, the smallest ρ at which certification becomes reliable, and convert it to units of active Sharpe using the oracle calibration of E2, so the threshold can be stated as the weakest signal, in Sharpe units, that the loop reliably certifies.

6.3 E2: search amplification

This experiment measures how much the search overstates a real signal. For each ρ we know the reference score: what a selector would earn if it ranked names by the planted feature alone and used nothing else. We compare the champion’s cross-validated score to that reference. The champion routinely scores above it. The reason is that the search does not stop at the planted feature; it also fits the three noise features, and whatever fit it finds in them adds to the measured score even though it is not real. The size of this gap quantifies the upward bias of any score produced by a

search, and it is the direct reason a champion’s score cannot be read at face value and must instead be compared to the bar.

Inputs. The per-run champion scores from E1, and for each ρ an oracle reference: the active Sharpe earned by a selector that each day ranks the seven names by `feat_c` alone and holds the top name, computed on the full sample and averaged over three independent noise draws. The reference is what the planted feature is worth on its own, with no search involved.

Procedure. For each ρ the 30 champion scores from E1 are placed next to the oracle reference and the difference is measured. No new searches are run; this is a re-analysis of the E1 output that isolates one quantity, the gap between what the search reports and what the planted signal is worth.

Outputs and analysis. We report the oracle reference and the champion scores side by side across ρ . A positive gap at a given ρ measures how much score the search extracted from the three noise features on top of the real signal. The conclusion is drawn by comparing three numbers at each ρ : the oracle reference, the champion score, and the bar. Champions exceeding the oracle reference is expected overfitting; the question the experiment settles is whether the bar accounts for that excess, so that the certification decision can remain anchored to the bar while the raw score is treated as biased upward.

6.4 E5: bar misspecification

This experiment makes a deliberate error in order to measure its cost. The correct procedure computes the bar with the same family of rules the search uses. Here the search uses the flexible nonlinear family, but we compute the bar from the simple linear family instead. The linear family is worse at fitting noise, so its bar is much lower. We then take the champions the nonlinear search finds on pure noise, certify them against this too-low bar, and count how many pass. Comparing that count with the count under the correctly matched bar isolates the false-positive rate caused purely by the mismatch. The result shows what goes wrong when the statistical test is borrowed from a weaker search than the one actually run.

Inputs. Pure-noise feature sets ($\rho = 0$) and weak-signal sets ($\rho = 0.15$); the nonlinear searcher with its usual budget of 200 candidates; and both bars from E3, the matched nonlinear bar of +0.844 and the mismatched linear bar of +0.408.

Procedure. The nonlinear search is run 30 times at each ρ , exactly as in E1, and each run’s champion is kept. Each champion is then judged twice, once against the matched bar and once against the mismatched bar. Nothing else differs between the two judgments: the champion, the data, and the score are identical, and only the threshold changes, so any difference in outcomes is attributable to the threshold alone.

Outputs and analysis. For each ρ and each bar we report the number of champions out of 30 that pass. At $\rho = 0$ every pass is a false positive, so the difference between the two counts is the false-positive rate caused purely by the mismatch. The experiment turns the same-class condition of Section 5.1 into a measured quantity: if the two counts were similar, matching the bar to the search would be a technicality, and a large difference means the family used to compute the bar is a correctness requirement of the method, not a preference.

6.5 E4: search-budget dependence

This experiment varies one quantity, the number of candidate strategies the search may try, from ten up to ten thousand, with everything else unchanged. More attempts on signal-free data produce higher best-on-noise scores, so the bar is recomputed at every budget, and we measure how fast it rises. At each budget we then repeat the E1 measurements: the false-positive rate on pure noise and the detection rate at fixed signal strengths. The question is whether a large enough number of attempts eventually produces noise champions that pass even the recomputed bar. The experiment answers it over three orders of magnitude of budget.

Inputs. Pure-noise features and planted-signal features at $\rho = 0.15$ and $\rho = 0.20$; the nonlinear family; and budgets $B \in \{10, 30, 100, 300, 1,000, 3,000, 10,000\}$.

Procedure. Running a separate search at every budget would waste computation, so the design is cumulative. On the noise side, each of 100 scrambles draws a fresh pool of 10,000 random formulas, evaluates all of them on the scrambled data, and records the running best score after the first 10, 30, 100, and so on up to 10,000 candidates; the running best after B candidates is exactly what a search with budget B would have found. The bar at budget B is the 95th percentile of these running bests across the 100 scrambles. On the signal side the same cumulative design is applied to 30 independent pools per ρ on the planted-signal data, giving one champion score at every budget for every pool.

Outputs and analysis. For each budget we report the recomputed bar, the certification frequency at $\rho = 0$ out of 30 against the budget-matched bar, and the certification frequencies at $\rho = 0.15$ and $\rho = 0.20$. Three outcomes are possible, and the data decides among them: the bar rises with budget and false positives appear anyway, meaning budget defeats the correction; the bar rises and detection of real signal collapses, meaning budget destroys power; or the bar and the real-signal champions rise together and both error rates hold, meaning the correction tracks the budget over the tested range.

6.6 E7: LLM versus mechanical search

The experiments above use the mechanical searchers, so this one checks whether the language model behaves differently. We put the model back into the loop as the component that writes the candidate strategies and run the search at three signal strengths, three runs each. The champions are scored against the same fixed bar as the mechanical runs, so the comparison is like for like. We then compare the signal strength at which the model starts reliably passing the bar with the strength at which the mechanical searchers do. Because model runs are slow and costly, this arm has far fewer repetitions; the mechanical results carry the statistics, and this experiment serves as a consistency check of the model against them.

Inputs. Planted-signal features at $\rho \in \{0, 0.15, 0.20\}$; the language model in place of the mechanical searcher; and the fixed nonlinear bar of +0.844 from E3.

Procedure. Each run executes the full loop of Section 4 with the model writing the candidates: 12 proposal rounds, each producing one new scoring function from the task description and the scored history of earlier candidates, with differential evolution fitting each candidate’s parameters under a budget of 60 evaluations and purged cross-validation scoring it. Three runs with different seeds are executed at each signal strength. The run’s own software also computes a bar over the candidates the model happened to propose; that per-run bar is computed over a smaller population

and is generally lower, so for a like-for-like comparison every champion is certified against the fixed E3 bar, the same threshold the mechanical runs face.

Outputs and analysis. For each signal strength we report the three champion scores and how many exceed the fixed bar, next to the corresponding mechanical certification frequency from E1. The comparison supports only a coarse conclusion: with three runs per point, a 3-of-3 result still has a 95% interval extending down to 29%, so agreement with the mechanical threshold can be observed but equivalence cannot be established. Two features of the design limit what the model can use. It never sees prices or feature values, only the scored history of its own candidates, and the features are opaque noise series with no names or meaning, so the model’s knowledge of markets gives it no usable advantage here; a test in which the features have meaning is the benchmark of Section 9.6.

7 Results

7.1 The null-max bar (E3)

Over 1000 sign-flip scrambles, the 95th-percentile bar is +0.844 for the high-capacity nonlinear class and +0.408 for the bounded linear class. The gap reflects both capacity and search mechanism: the linear class is fit by continuous differential evolution and the nonlinear class by discrete enumeration of trees, so the two are not a controlled capacity comparison. The classes also differ in how fast the estimate settles. The nonlinear bar is stable beyond roughly 100 scrambles; the linear bar keeps rising as scrambles increase, with a growing tail maximum, so a small scramble count under-estimates a low-capacity bar. Experiments E1, E5, E7, and E12a use the 1000-scramble bar; the capacity sweep (Section 7.5) recomputes the bar at 100 scrambles per budget over the nonlinear class, which the stability result shows is adequate.

7.2 Power curve and false positives (E1)

Table 2 gives the certification rate against injected signal, 30 seeds per point, each class certified against its own bar.

Table 2: Certification power curve (30 seeds per point). “Oracle” is the active Sharpe a perfect selector on `feat_c` attains at that ρ , a descriptive full-sample quantity used only to set signal strength and never seen by the search or certification. The certification frequency on noise ($\rho = 0$) is 0 of 30 for both classes.

ρ	oracle Sharpe	nonlinear $P(\text{clear})$	median champ	linear $P(\text{clear})$
0.00	−0.67	0.00	+0.44	0.00
0.10	0.00	0.03	+0.68	0.00
0.15	+0.30	0.43	+0.83	0.10
0.20	+0.66	1.00	+1.10	0.70
0.25	+0.95	1.00	+1.40	0.80
0.30	+1.23	1.00	+1.77	1.00
0.40	+1.66	1.00	+1.96	1.00
0.50	+2.06	1.00	+2.17	1.00
0.70	+2.81	1.00	+2.79	1.00

Figure 5 plots the two power curves. The boundary is sharp. Nothing clears at an oracle Sharpe at or below zero, and clearing becomes reliable for the nonlinear class from $\rho = 0.20$, where the oracle Sharpe is about 0.66; the linear class reaches reliable clearing at $\rho = 0.30$. The observed

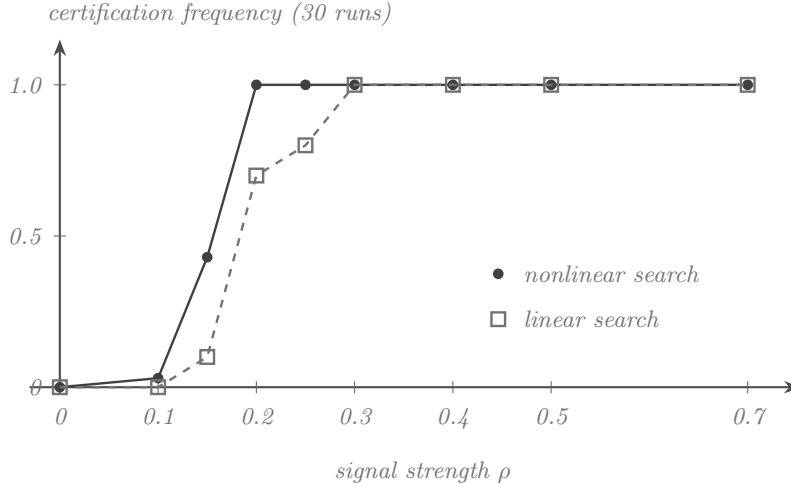


Figure 5: The power curves of Table 2. Each point is the fraction of 30 runs whose champion cleared the family’s own bar at that signal strength. The nonlinear search becomes reliable at $\rho = 0.20$ and the linear search at $\rho = 0.30$; neither certifies pure noise at $\rho = 0$.

certification frequency on noise ($\rho = 0$) is 0 of 30, a 95% Clopper-Pearson upper bound of 11.6%; rates below are observed frequencies across seeds, not asymptotic probabilities. Constraining the class to linear does not buy sensitivity here: the nonlinear class certifies at $\rho = 0.20$ and 0.25 more reliably than the linear class, because its champion overfits above its higher bar while the linear champion grazes its lower bar with more seed-to-seed variance. The two classes carry different bars, and neither dominates on detection power.

7.3 Search amplification (E2)

The champion’s cross-validated score routinely exceeds the oracle ceiling. At $\rho = 0.20$ the oracle is 0.66 while nonlinear champions reach about 1.10, because the loop fits noise features on top of the real signal. The amplification is why the operative hurdle is the bar near 0.6 to 0.8 and not the oracle ceiling: the champion must beat what a same-capacity class extracts from noise, which it does once real signal is present.

7.4 Same-class ablation (E5)

The certification rule of Section 5.1 requires the bar to be computed by running the same search that produced the champion. This experiment measures what that requirement is worth by violating it once, deliberately, and counting the consequences.

The two search families measured in E3 carry different bars on identical data. The nonlinear family can combine four features through an unrestricted expression tree, so it has many ways to fit noise, and its best-on-noise score is correspondingly high: its bar is +0.844. The linear family, limited to four weights, has far fewer ways to fit noise: its bar is +0.408. Neither number measures signal. Each is the 95th percentile of the best score that family’s search extracts from scrambled, signal-free returns, so each is the threshold that family must beat before its champion means anything.

Now take the 30 champions the nonlinear search found on pure noise ($\rho = 0$). Their median score is about +0.44, which is ordinary noise fitting for this family and sits well below the family’s own bar. Judged against the matched nonlinear bar of 0.844, 0 of 30 pass (95% upper bound 11.6%). That is the correct outcome, because there was no signal to find. Judged against the linear bar of

0.408 instead, 20 of 30 pass (95% interval roughly 47 to 83%). The champions, the data, and the scores are identical in the two judgments; only the threshold changed. Twenty signal-free strategies would be certified as carrying real signal, because the test asked whether each champion beat what a four-weight linear rule can extract from noise, while the search that produced the champion was free to build arbitrary nonlinear expressions and therefore extracts far more from noise than a linear rule does.

This error is easy to commit in practice. The bar might be estimated with a cheap, simple search to save computation while the production search is more flexible; the search might be upgraded later, with more operators or more features, without recomputing the bar; or a threshold might be borrowed from a published study whose search differed from one’s own. In each case the certified output contains noise champions at whatever rate the capacity gap produces, two-thirds here for a single step down in capacity on this test-bed.

The direction of the mismatch determines the kind of failure. A bar computed over a class larger than the search is too high, which costs detection power but keeps the false-positive guarantee. A bar computed over a class smaller than the search is too low, and the guarantee is gone. The same-class condition is therefore a correctness requirement of the method, not a preference.

7.5 Search capacity (E4)

We hold the class fixed and vary its search budget B , recomputing the noise bar (100 scrambles per budget) and the certification rate at each budget (Table 3).

Table 3: Capacity sweep for the nonlinear class, 30 pools per point. The noise bar rises with budget, while the certification frequency on noise ($\rho = 0$) stays at 0 of 30 (95% upper bound 11.6%) up to ten thousand candidates.

budget B	noise bar q_{95}	$P(\text{clear}) \rho=0$	$P(\text{clear}) \rho=0.15$	$P(\text{clear}) \rho=0.20$
10	0.59	0.00	0.77	0.77
30	0.70	0.00	0.17	1.00
100	0.79	0.00	0.47	1.00
300	0.88	0.00	0.37	1.00
1000	0.95	0.00	0.53	1.00
3000	1.04	0.00	0.53	1.00
10000	1.20	0.00	0.37	0.97

Figure 6 plots the sweep. Three results follow. The noise bar rises steadily with budget, from about 0.6 to about 1.2 over three decades, with no plateau. The certification frequency on noise ($\rho = 0$) stays at 0 of 30 at every budget, including ten thousand candidates, because the bar is the 95th percentile of the same search on scrambled returns and so tracks the search. For a threshold signal ($\rho = 0.15$) the champion score and the bar rise together, so the certification rate stays near one half at every budget, while a clear signal ($\rho = 0.20$) survives to the largest budget.

The naive expectation that a high-capacity search slips past the bar on noise does not hold up to ten thousand candidates. The price of capacity is a rising bar and reduced sensitivity, not false certification. We flag this as encouraging but preliminary. The class here is the finite-capacity random-tree family, a high-capacity proxy rather than the unbounded code-writing class of Eq. (3); the unbounded case rests on the VC argument and the three LLM seeds of Section 7.6, not on measurement. The result also holds in part by construction, since a same-class bar with a valid null fixes the false-positive rate at any capacity. It is untested beyond ten thousand candidates, where the VC argument still predicts a loss of power for a fixed weak signal as the bar grows without

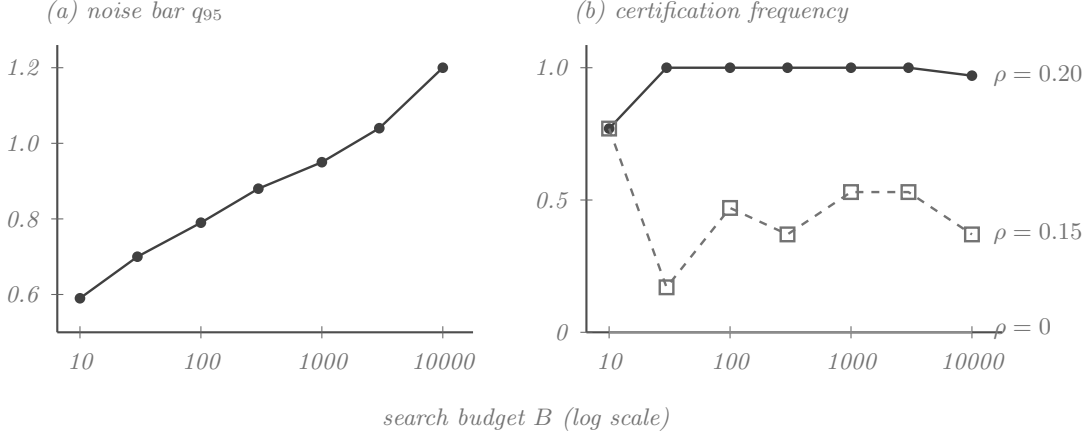


Figure 6: The capacity sweep of Table 3. (a) The bar measured on noise rises steadily as the budget grows from 10 to 10,000 candidates. (b) Certification frequency against the budget-matched bar: zero at $\rho = 0$ at every budget, at or near one at $\rho = 0.20$, and fluctuating with no trend at the threshold strength $\rho = 0.15$.

bound, consistent with the lockstep we observe. And the sign-flip null tests directional signal, which is valid for this long-only selection task, where all profit and loss is directional, but need not hold for a class with a non-directional overfitting channel.

7.6 LLM and mechanical search (E7)

The LLM arm uses the Claude Code subagent to propose each structure. We run 3 seeds at each of three signal levels and certify the champions against the fixed nonlinear bar (0.844) for comparison with mechanical E1. Table 4 reports the result.

Table 4: LLM champions certified against the fixed nonlinear bar (0.844), with the mechanical nonlinear rate at the same ρ for comparison.

ρ	LLM clears	LLM champion scores	mechanical $P(\text{clear})$
0.00	0/3	0.52, 0.22, -0.05	0.00
0.15	1/3	0.82, 0.91, 0.36	0.43
0.20	3/3	1.22, 1.02, 1.40	1.00

Figure 7 plots the champions against the bar. The LLM recovers signal at the same threshold as mechanical search; its champions anchor on `feat_c`, the true predictor. In this three-seed experiment we observe no evidence of an advantage for the LLM over mechanical search of the same vocabulary. The three-seed intervals are wide (for example 3/3 corresponds to a 95% lower bound near 29%), so this is an absence of evidence, not evidence of equivalence; the mechanical searchers are the statistical benchmark and the LLM arm is a proof-of-concept. The comparison is also stacked toward the mechanical side, because the four opaque features give a semantic prior almost nothing to use; a nameable-feature test (Section 8.10) is where the LLM’s prior could matter. This supports only the empirical statement, that the LLM did not outperform mechanical search here, and not the stronger theoretical claim about hypotheses outside the model’s distribution, which we argue separately in Section 8. (The search driver otherwise computes a per-run bar over each run’s own, smaller population; we note this and certify against the fixed bar for the comparison.)

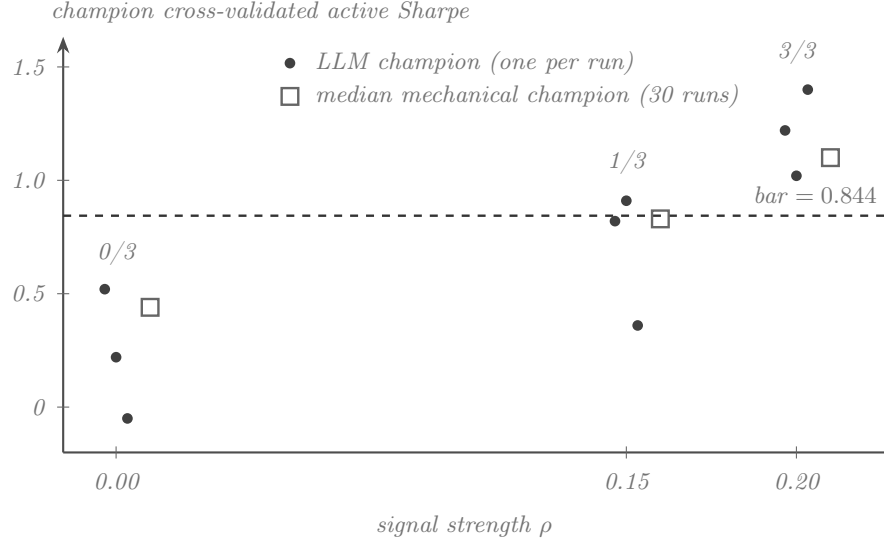


Figure 7: The E7 comparison of Table 4. Each filled circle is the champion score of one LLM run; each open square is the median champion of the 30 mechanical runs at the same ρ (E1). The dashed line is the fixed nonlinear bar of 0.844; points above it are certified. The LLM clears in 0 of 3 runs at $\rho = 0$, 1 of 3 at $\rho = 0.15$, and 3 of 3 at $\rho = 0.20$, against mechanical certification frequencies of 0.00, 0.43, and 1.00.

7.7 Implementation

Each search runs 12 mutation iterations, seeded with simple feature-combination structures. A candidate’s parameters are fit by differential evolution (population 8, a budget of 60 evaluations, mutation weight sampled in $[0.5, 1.0]$); the mutation prompt uses temperature 1.0. The prompt is a fixed task contract plus the scored history of prior structures, strongest first, and the model returns one new structure. The LLM arm routes each mutation to the Claude Code subagent over a filesystem handoff. The model sees only the task contract, the four feature names, and prior structures with their cross-validated scores; it never sees prices, returns, or feature values. The mechanical searchers use no model and share the rest of the stack.

7.8 Findings

The experiments were built to answer one question: does the loop work as a measurement tool, so that a certification from it can be trusted? The answer is yes, within the limits stated at the end of this subsection, and the measured facts are the following.

The loop detects real signal, and the detection threshold is an oracle active Sharpe of about 0.66. At $\rho = 0.20$, where the planted feature on its own is worth an active Sharpe of about 0.66, the nonlinear search certified in 30 of 30 runs, and it continued to certify at every higher signal strength. At $\rho = 0.15$ (oracle about 0.30) certification was unreliable, 13 of 30 runs; at $\rho = 0.10$ it occurred once in 30 runs; at $\rho = 0$ never. So under these settings the loop reliably identifies the strategy when the true signal in the supplied features is worth roughly 0.66 active Sharpe or more, is unreliable near 0.30, and finds nothing weaker. The linear search becomes reliable later, at $\rho = 0.30$ (oracle about 1.23), with 21 of 30 at $\rho = 0.20$ and 24 of 30 at $\rho = 0.25$: restricting the search to simpler rules did not lower the detection threshold in this setup.

The loop rejects noise. At $\rho = 0$ the nonlinear search never certified against its matched bar, 0 of 30 runs, which bounds its false-positive rate below 11.6% at 95% confidence; the linear search also gave 0 of 30. The same zero count held at every search budget tested, from 10 to 10,000 candidates.

The bar is measurable and stable, and it belongs to the search, not to the data. The nonlinear family’s bar is +0.844 and its estimate is stable from roughly 100 scrambles. The linear family’s bar is +0.408 and was still drifting upward at 1,000 scrambles, so a low-capacity family needs more scrambles, not fewer. The two bars differ by roughly a factor of two on identical data, which is the empirical content of the claim that the certification threshold is a property of the search procedure.

Champion scores are inflated and cannot be read at face value. At $\rho = 0.20$ the planted feature alone is worth about 0.66, but the median certified champion scored +1.10; the excess is fit to the three noise features. The certification decision is unaffected, because the bar is built from the same behavior on noise, but the reported Sharpe of any searched-for champion overstates the signal underneath it.

A mismatched bar destroys the guarantee. The bar measures how well a particular search fits noise, so it must be computed by the same search that produced the champion. The nonlinear search fits noise up to a bar of +0.844; the linear search only up to +0.408. When the 30 pure-noise champions of the nonlinear search were judged against their own bar, 0 of 30 passed. When the same champions, with the same scores on the same data, were judged against the lower linear bar, 20 of 30 passed, and every one of those 20 is a certified strategy that contains no signal. Two-thirds false positives from this single bookkeeping error, using a bar computed with a simpler search than the one actually run, is the largest failure mode measured in this paper.

Raising the search budget did not break certification in the tested range. From 10 to 10,000 candidates the bar rose from 0.59 to 1.20 and false positives remained 0 of 30 at every budget. At $\rho = 0.20$ detection held at 30 of 30 at every budget of 30 or more (23 of 30 at the smallest budget, 29 of 30 at 10,000). At $\rho = 0.15$ the champion scores and the bar rose together and certification fluctuated between 5 and 23 of 30 with no trend as the budget grew: more search neither rescued nor buried a threshold-strength signal.

The language model matched the mechanical searchers. Against the same fixed bar the LLM certified 0 of 3 runs at $\rho = 0$, 1 of 3 at $\rho = 0.15$, and 3 of 3 at $\rho = 0.20$, the same threshold the mechanical search shows (certification frequencies 0.00, 0.43, 1.00). Three runs per point cannot establish equivalence, but there is no evidence of an advantage.

Scope. These statements hold for this synthetic test-bed, with in-sample certification against the null-max bar, the directional sign-flip null, and the stated families and budgets. They establish that the loop is a working detector of signal that is present in its supplied features. Whether a given real feature set contains such signal is a separate question, one the loop can test but cannot answer on its own, and the loop cannot supply the predictive feature itself, which is the subject of the sections that follow.

8 The limits of automated alpha search

LATSS searches and certifies strategies over a feature vocabulary that a human supplies, and whether it produces alpha depends on whether those features predict. The question is whether the vocabulary itself can be automated, whether an LLM system can invent new alpha. By inventing new alpha we do not mean producing a factor formula nobody has published, which a search system does easily. We mean originating a new hypothesis about where returns come from: a variable, mechanism, or relationship outside the space the system was given. This is a stronger property than producing outputs that people judge novel, which we take up in Section 8.4.

8.1 Search and abduction

The distinction that matters in this section is between two acts. *Search* is picking the best candidate out of a given collection: there is a set of possible strategies, a score for each, and the searcher returns the highest-scoring one it finds. This is what LATSS does, what FunSearch does, and what LLM factor-mining systems do. *Abduction* is producing a candidate the given collection does not contain: extending the collection when nothing inside it explains the phenomenon. Search presupposes that the collection worth searching has already been supplied; abduction is what supplies it.

Now apply the distinction to a language model used as a proposer. The model was trained on a body of text and code, and everything it proposes is some recombination or re-expression of what that training material contains. Call the set of everything reachable this way the model’s *reachable set*, written $\mathcal{C}(D)$, where D stands for the training material. The term is a conceptual abstraction, a way of talking about what recombination can reach, not a set anyone can measure.

Two consequences follow. First, if the collection being searched lies inside the reachable set, then every proposal, and therefore every champion, lies inside it too. Novelty of that kind is routine: the model emits formulas nobody has written down, but each one is a recombination of what it was trained on, which is interpolation, not invention. Second, enlarging the search does not escape the set. Asking the model for new variables, new datasets, or new ideas enlarges the collection being searched, but the suggestions still come from the reachable set, each a recombination of hypotheses already present in the training material. The abductive target is, by definition, a hypothesis the reachable set does not contain, and no amount of searching inside the set produces one. This is the content of Zahavy’s *LLMs Can’t Jump* [2], and it agrees with the finding that learning in high dimension is interpolation rather than extrapolation [7].

8.2 FunSearch

The strongest apparent counterexample is FunSearch [1], where an LLM coupled to an evaluator found previously unknown constructions. The human supplies the problem, the evaluation function, and the representation, and the system then generates on the order of a million candidates and keeps the best. FunSearch shows that an LLM can participate in discovery. It does not show the LLM deciding what should be discovered. It did not conclude that a different representation was needed and invent it; it searched very well inside a space a human defined.

8.3 Creating a search space

Nobody handed Einstein a defined problem to search. He saw that the existing framework was inadequate and proposed a new premise, the equivalence principle, which changed what a valid explanation could look like and created a new search space (Figure 8). An LLM stands on the

shoulders of every giant at once, but standing on shoulders lets it see further along a view that already exists. A new hypothesis is a shoulder no one has built.

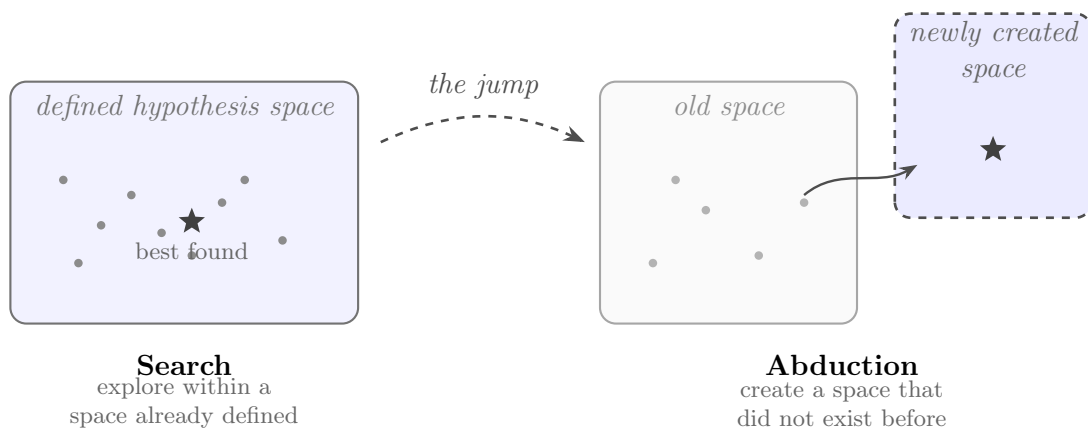


Figure 8: Search explores within a hypothesis space already defined. Abduction creates a space that did not exist, the step an LLM has not been shown to take.

8.4 Evidence on machine invention

Two questions should be kept apart. The first is whether LLMs can produce outputs that people judge novel. They can. In a controlled study with more than one hundred NLP researchers, Si, Yang, and Hashimoto found that LLM-generated research ideas were rated more novel than those of human experts, though less feasible [3]. A system named The AI Scientist, published by Sakana AI, automates much of the research loop, generating ideas, writing and running code, and producing manuscripts, and reports experimentally evaluated contributions [4]. This is novelty of output.

The second question is whether an LLM has independently originated a new explanatory hypothesis whose conceptual content was not already present in its training distribution, its prompt, its search vocabulary, or a human-designed experimental frame. Here the evidence is thin. The reported novelty is measured against a literature corpus or human judgment, and the autonomous systems run inside substantial human scaffolding. An independent evaluation of that system found that it often labeled established techniques as novel and that many of its experiments and manuscripts were flawed [5]. In a controlled molecular-genetics setting, Ding and Li report that a generative model made incremental discoveries but not fundamental ones from scratch and did not produce genuinely original hypotheses [6]. Zahavy frames the same gap as a missing abductive step and takes Einstein’s route to general relativity as the case [2].

The distinction is between novelty of output and novelty of hypothesis. The first is increasingly demonstrated. The second remains open, and to our knowledge no published result has established it with provenance showing the hypothesis was not inherited from the training data or the search frame. LATSS does not settle the question. Section 8.10 tests one part of it: when the predictive feature is withheld from the vocabulary, more search does not recover it, because search cannot recover information absent from the space it is given. An LLM may find a new solution inside a financial hypothesis space without finding a new reason the market behaves as it does; the first is a search problem, the second is the invention problem.

8.5 Financial alpha

Everything is decided before the search: which data is relevant, the prediction target, the horizon, the available transformations, the evaluator, and the tests of success. A factor miner handed RSI, volume, volatility, and momentum can find an expression nobody has written, and it remains a construction from the given vocabulary. When the source of an anomaly is a variable that is not in the vocabulary, not among the available transformations, and not in the database, searching the given space harder does not surface it, because the missing ingredient is outside the space. More compute, more agents, deeper tree search, and a larger context window do not change this.

8.6 Proposing hypotheses

One response is to move the search up a level and ask the model for new variables and mechanisms, which it supplies fluently: filing sentiment, supplier networks, options positioning. Each proposal is an idea the model read somewhere, a recombination of hypotheses already written down. The proposed variable is new to the reader, not to the world. Generating hypotheses is still search; the jump is inventing one the space did not contain.

8.7 Two ceilings

Alpha resists automated discovery for two independent reasons. The cognitive ceiling is the inability to produce a hypothesis outside $\mathcal{C}(D)$ (Section 8.1), which is intrinsic to the model. LeCun’s critique that LLMs have no world model [7, 8] is a separate limitation that scaffolding can lift: LATSS supplies a grounded world model externally, since the features are measurements of the world and the backtest is the reality check, which is why the loop of Sections 3–7 works. Grounding tells the model whether a hypothesis it already holds survives the data; it does not generate a hypothesis outside $\mathcal{C}(D)$. The market-structure ceiling is that alpha is adversarial and perishable and its evaluator is causally ambiguous: a successful backtest does not say why a factor predicted, whether mechanism, correlation, hidden risk, regime, leakage, cost, or crowding. A new mechanism is necessary but not sufficient for durable alpha, and the bar addresses overfitting within the search, not the missing hypothesis.

8.8 Levels of novelty

Novelty covers four distinct achievements (Table 5), which lets a claim be held to a matching standard of evidence. The first two can be reached by search within a supplied space; the last two require evidence that the predictive premise was not already present in it.

The claim that current systems operate at the first two levels and have not reached the last two is falsifiable. It is refuted by a system that, given raw data and no supplied vocabulary, repeatedly originates durable, economically intelligible returns no source in its training data specified, with a human or oracle positive control showing the step was achievable. We know of no such demonstration. Reviewers rate LLM ideas as more novel than their own [3], which measures how novel an idea looks rather than whether it is real.

8.9 Current systems

Table 6 places representative systems by search space and novelty level. The level is a qualitative classification we argue from each system’s design, not a bibliographic fact. Each recombines a human-defined primitive vocabulary, and we are aware of no published demonstration of one originating

Table 5: Levels of novelty in automated alpha discovery. The last two require a hypothesis outside the model’s reachable set $\mathcal{C}(D)$.

level	definition	evidence that would demonstrate it	state
Syntactic	a formula no one has written	any run emitting a novel expression	routine
Solution	a new construction for a defined problem	FunSearch cap-set and bounds [1]	demonstrated
Hypothesis	a new explanation for an unexplained phenomenon	a system originating a driver no source in D contains, with a positive control	not demonstrated
Economic	a new, durable, uncrowded source of returns	out-of-space edge surviving forward, after costs and crowding	none convincing

an economic primitive no one specified. AlphaAgent’s penalty on syntax-tree similarity to crowded factors [14] is itself evidence that, unregularized, LLMs regenerate known factors.

Table 6: Representative LLM alpha systems by search space and novelty level (as in Table 5); the level is a qualitative classification argued here.

system	primitive vocabulary / search space	machinery	level
LATSS (this work)	given indicators to scoring rule	DE, null-max bar	2
AlphaAgent [14]	factor formulas over raw price/volume	multi-agent, AST penalty	2
Automate Strategy Finding [15]	factor/strategy formulas over raw inputs	multi-agent, RL	2
CogAlpha [16]	factor formulas over raw inputs	LLM, tree search	2
QuantaAlpha [18]	factor formulas over raw inputs	multi-agent	2
AlphaLogics [19]	logic and formula factors	search, verify	2
EFS [20]	evolutionary factor formulas	evolutionary search	2
FunSearch [1] (math)	programs for a defined problem	LLM, exact evaluator	2

8.10 Withholding the predictive feature (E12a)

The vocabulary boundary is measurable. We inject the signal into `feat_c` as in Section 7.2, then overwrite the searcher’s `feat_c` with independent noise, so the predictive feature is absent from the vocabulary. Across ρ from 0 to 0.70, 30 seeds each, the certification rate is zero at every signal strength, with the champion median near 0.43 against a bar of 0.844. Section 7.2, with `feat_c` present, recovers reliably from $\rho = 0.20$. Recovery is governed by whether the predictive feature is in the vocabulary, not by signal strength or search effort. We state the limit of this result directly: a mechanical searcher fails the same way, so it demonstrates the vocabulary boundary and not an LLM-specific inability. A test that isolates the LLM-specific step would need a semantically nameable driver together with a human or oracle positive control, which the opaque synthetic driver

here cannot provide, and we leave it to future work.

8.11 The ceiling

The ceiling is the line between searching a hypothesis space and creating one, and current alpha agents sit on the near side. LATSS is the alpha analogue of a coding agent: it generates, validates, and recombines strategies from a human-supplied vocabulary under human supervision, scored against the backtest much as a coding agent is scored against its tests. Both search and certify over known primitives, and neither invents a new paradigm. Crossing the line would mean a system that, given raw data and no predefined vocabulary, repeatedly finds durable, economically intelligible returns no one told it to look for. We are aware of no such demonstration.

9 Toward abduction of alpha

The ultimate goal is a system that discovers new alpha, which is a special case of scientific discovery: a durable economic edge rests on a new explanation of why a market behaves as it does. To aim at that goal we ask how human discovery works, locate where a language model can and cannot contribute, and give a concrete benchmark that would make progress measurable.

9.1 The structure of scientific discovery

Accounts of discovery in the philosophy of science converge on a two-part structure. Peirce separated three modes of inference: deduction draws out consequences, induction generalizes from instances, and *abduction* generates an explanatory hypothesis to account for a surprising observation; only abduction introduces a new idea, and Peirce held that the abductive suggestion is not reached by algorithm [29]. Hanson made abduction a logic of discovery and stressed that inquiry starts from an anomaly, an observation the current frame does not predict [33]. Popper denied that there is any logic for producing the hypothesis at all: science proceeds by *conjectures and refutations*, bold guesses exposed to attempted falsification, with the generation of the conjecture left to the scientist and only its testing governed by method [30]. Kuhn distinguished normal science, puzzle-solving inside an accepted paradigm, from revolutionary science, where the paradigm itself is replaced [31]; Lakatos recast this as research programmes with a protected hard core [32]; and Schurz catalogued the kinds of abductive guess, from factual to theoretical-model abduction [34]. Computational work matches the division: the BACON system of Langley and Simon rediscovered quantitative laws by heuristic search over a supplied representation, which is discovery at the level of a new construction inside a given vocabulary rather than a new vocabulary [35].

Two things recur. Discovery is conjecture plus test, iterated at scale: a guess is proposed, often triggered by an anomaly, and is then kept or discarded by confronting it with evidence, and the rare surviving guess is the discovery. And the creative content is the conjecture, which every account treats as non-algorithmic, while the testing is the part method can systematize.

9.2 The language model in the discovery cycle

This division locates the language model precisely, and it depends on what a transformer is. At inference a trained transformer is a fixed function from a context to a distribution over continuations. Its weights are frozen, and it carries no state between calls; the only memory it has is what is placed in its context window. Persistence, accumulation of results, and iteration are therefore not properties of the model but of the agent built around it.

The transformer at inference. A decoder-only transformer maps a token sequence through an embedding, a stack of self-attention and feed-forward blocks with residual connections, and a final projection and softmax, producing a probability distribution over the next token. Generation is autoregressive: a token is sampled, appended to the context, and the pass repeats. The parameters do not change during this process, and the key-value cache that makes generation efficient is internal to a single sequence and is discarded between calls. In-context learning does not change the picture: a prompt alters behavior by conditioning the same fixed function on a longer input, not by updating any weight, so the apparent learning is a reweighting of the existing distribution rather than the acquisition of anything new. The weights are fixed, so the conditional distribution the model samples from is set by training and by the context it is given, and its reach is, as a conceptual abstraction, the recombination closure $\mathcal{C}(D)$ of Section 8.1. Sampling can produce token sequences no one has written, which is the syntactic and solution novelty of Table 5, yet sampling from a fixed model provides no explicit mechanism for creating a hypothesis whose relevant conceptual content is absent from the model’s learned representation. Raising the temperature makes low-probability continuations more likely relative to one another but adds no region the distribution never represented. Chain-of-thought and tool use lengthen and externalize the computation, and each added step is another conditioned forward pass, so they deepen deduction and search without moving outside $\mathcal{C}(D)$. This is the mechanism behind the categorical claim below: the single operation available to the model, sampling a continuation from a fixed conditional distribution, is not the operation that creates a hypothesis outside that distribution.

Mapping this onto the discovery cycle, the model supplies the conjecture step, sampling a candidate from its reachable set $\mathcal{C}(D)$ (Section 8.1). The agent supplies the rest: the anomaly that prompts a conjecture, the memory of what has been tried, and the test that refutes or retains it. In LATSS the evaluator and the null-max bar are that test, a mechanized refutation step. The paradigm of this paper is thus conjecture and refutation run at scale: the loop industrializes the guessing-and-testing that the philosophy of science describes, with the model as the guess generator and the bar as the refutation.

The framing explains both the value and the limit. The test is solved, which is the contribution of Sections 3 to 7; an agent can now generate and refute conjectures far faster than a human. But a stateless sampler over $\mathcal{C}(D)$ proposes guesses from inside its own distribution, so scaling the loop produces more conjectures of the same kind and more tests, which is Kuhn’s normal science. The abductive jump is a conjecture $\mathcal{C}(D)$ does not contain, and no amount of sampling from a distribution returns a point outside its support. Progress toward new alpha is therefore progress on the conjecture step, not the test.

Invention is not inference. The point is categorical rather than a matter of scale. An inference engine maps an input to an output within a learned distribution; a jump is the creation of a hypothesis outside that distribution, which is a different kind of act. Expecting an inference engine to jump is a category error, and the failure of current models to invent is not a defect to be patched but a consequence of what inference is. Whatever performs the jump must sit outside the inference step. On this reading the language model’s role is to *assist* invention, not to perform it: it is a fast generator of conjectures inside its reach and a competent executor of the search and coding that discovery also requires, while originating a genuinely new hypothesis, and the agent architecture that might one day carry it, lies outside the model. LATSS is built on this division. The model proposes and the surrounding loop tests; we neither ask nor expect the model itself to originate the alpha, and a system that one day does so will have placed the generative step in the agent that wraps the model, not in the model.

9.3 Invention as an agent-level capability

The model is doing its job, and invention is the agent's. The failure to invent is not a defect of the model to be patched, so the research target is not a better inference engine but an agent that invents, with the model as one component. Operationally that agent is a conjecture-and-refutation loop of two stages, a proposal stage that generates a candidate hypothesis and a validation stage that keeps or discards it, iterated. LATSS is already this loop, and its validation stage, certification against the null-max bar, is the part this paper solves. The underdeveloped half is the proposal stage for genuinely new hypotheses.

Cast as search, there is a space H of hypotheses, and inquiry draws candidates from H and tests them. The true hypothesis may lie inside the supplied space or outside it. Inside, this is ordinary search and the model helps. Outside the supplied vocabulary, outside $\mathcal{C}(D)$, no search over H reaches it, which is the ceiling of Section 8, and the agent's task then also includes enlarging H .

The harder part, even with H fixed, is the proposal heuristic: a prior over H that says which conjectures are worth testing, since enumerating H is infeasible. Peirce called abduction the economy of research, locating the value in choosing which guess to try [29]. Here the model earns its place, because its training distribution is exactly such a prior, ranking hypotheses by plausibility, so the model is a usable proposal heuristic. But that prior is centered on what is common in training, the known and crowded ideas of $\mathcal{C}(D)$, which is why these systems regenerate known factors and why penalties on similarity to crowded factors are needed [14]. The heuristic exists; it points at the wrong region for new alpha.

This sharpens the open problem. A system that invents needs two things the model does not supply: a way to enlarge the hypothesis space, and a proposal heuristic that favors promising but uncrowded hypotheses rather than the crowded center of the training distribution. The available levers re-aim the existing prior without leaving it: handing the model the space and sampling from it, conditioning proposal on an anomaly so the misfit directs where to look, and regularizing away from crowded factors. Each improves the proposal within $\mathcal{C}(D)$; none makes it sample outside. The benchmark of Section 9.6 measures exactly the step they do not reach.

9.4 From the validated search loop to an invention loop

The results of Section 7 establish that the search loop works as a detector: it rejects noise, recovers signal above a measured threshold, and controls false positives through the bar. An invention loop would have the same two-stage shape, propose and then refute, so the natural conjecture is that the validated machinery extends: keep the evaluator and the bar, and replace the proposal stage with one that can enlarge the vocabulary. We cannot claim this, because no component of the proposal stage for invention has been demonstrated, by us or, to our knowledge, by anyone. We state it as the working conjecture of this paper and itemize, in Table 7, exactly which components carry over from the loop we validated and which are missing.

The table shows where the conjecture stands. The refutation half of the invention loop, evaluation and certification, is finished and measured; every open gap sits in the proposal half, the trigger, the conjecture of a new variable, and the means of verifying that a conjecture was a conjecture. That the gaps concentrate there is the thesis of this paper restated as an engineering inventory.

9.5 Alpha discovery as an inverse problem

The division of labor above has a precise counterpart in inverse-problem theory, a mature field in the physical sciences. The mapping is close enough that progress on solving inverse problems with

Table 7: Component-by-component gap between the search loop validated in this paper and the invention loop it suggests.

Component	Validated search loop	Invention loop	Gap
Hypothesis space	Fixed feature vocabulary supplied by a human	Must grow at run time: the agent requests measurements that were never supplied	Affordance and resolver exist only in the proposed benchmark (Section 9.6)
Proposal step	Mutate formulas over the given features, by model or by grammar	Conjecture a nameable new variable or mechanism	No demonstrated mechanism; the model’s prior points at crowded, known ideas
Trigger	None; the search runs unconditionally	An anomaly: residual structure the current features do not explain	The loop does not yet surface residuals as prompts
Evaluation	Backtest with purged cross-validation	Unchanged	None; carries over as validated (E1, E3)
Certification	Same-class null-max bar	The same bar, recomputed over the enlarged class after each accepted feature, plus a jump rate against a base rate	Bar recomputation is defined (Section 5.1); the jump metric is proposed, not built
Validation of the loop itself	Power curve on a dialed synthetic signal	Jump rate with a human positive control	Benchmark unbuilt; no measurement exists
Provenance	Not required	Must rule out that the named variable was retrieved from training data rather than conjectured	Unsolved

language models is, in effect, progress on automated alpha discovery, which makes that literature the most directly relevant precedent for the generative step LATSS leaves to the human.

The forward and the inverse problem. A forward problem maps a model, its parameters and structure, to observable consequences, and is usually well-posed: the output exists, is unique, and varies continuously with the input. The inverse problem runs the other way, inferring the model from observations, and is generically ill-posed in Hadamard’s sense: a solution may fail to exist, may not be unique, and may be unstable, so that a small change in the data forces a large change in the inferred model. Measurement noise sharpens all three difficulties. The classical responses are regularization and prior information, which restrict or weight the model space so that the inference becomes tractable.

Popper, Bayes, and Tarantola. The oldest tradition, from Gauss and Legendre, casts the inverse problem as optimization and seeks the single best-fit model, least squares being the criterion that follows from assuming Gaussian measurement errors [36]. Tarantola argues this is the wrong object. Observations, he writes, cannot produce models, they can only falsify them, so displaying one best model lends it a veracity its uncertainties do not warrant, and in a high-dimensional model space the posterior is typically multimodal, with whole subspaces of equivalent fits, so optimization to a point is not even well defined. His prescription is explicitly both Bayesian and Popperian: use all available prior information to generate models of the system, potentially without limit; for each, solve the forward problem, compare its predictions with the observations, and drop the models the comparison falsifies; the collection that survives is the solution [36, 37]. Passing from this prior collection of models to the surviving posterior collection is the Bayesian content, and the keep-or-drop test on each model is the Popperian content. Prior information is structural rather than optional, because the problem is underdetermined and the space of models is a high-dimensional haystack in which even finding one consistent model is hard.

The mapping to alpha search. The backtest is the forward map, taking a strategy to the returns it would produce. Recovering a strategy from observed returns is the inverse problem, ill-posed in Hadamard’s senses: many strategies fit a finite sample, the fit is unstable across samples, and the true mechanism may not lie in the considered class. Overfitting is this non-uniqueness. LATSS is Tarantola’s procedure in a specific form, step for step. Prior information is the model’s training distribution, the proposal heuristic of Section 9.3, which generates candidate strategies; the backtest solves the forward problem for each; and the null-max bar is the falsifying comparison, a candidate being dropped unless it beats what the same search extracts from data with the signal destroyed. The certified champions are the collection that survives falsification, so certification means not yet falsified, never confirmed, and the proper output is that surviving set rather than one model held as true. This is why we certify by falsification against a null rather than by best fit: in an ill-posed inverse problem the best fit is not identified, while survival against a correctly matched null is. It also restates the ceiling. A prior with no support on the true mechanism cannot place it in the posterior collection however much is falsified against, which is search inside a supplied space failing to reach a hypothesis outside it.

Solving inverse problems with language models. The active frontier here is LLM-driven symbolic regression and equation discovery, which is the inverse problem in its classic form: infer the governing equation of a system from data. LLM-SR treats an equation as a program and has the model propose equation skeletons from its scientific prior, with evolutionary search and numerical

fitting of the constants against the data [38]. SGA makes the split explicit as bilevel optimization, with the language model proposing discrete hypotheses and a differentiable simulation fitting the continuous parameters [39], the same separation LATSS draws between an LLM that writes structure and differential evolution that fits numbers. Benchmarks are consolidating the area [40]. Two lessons carry over to alpha. First, the architecture transfers: an LLM that proposes the model with a numerical layer that fits and a test that falsifies is the general shape of an LLM inverse-problem solver, and LATSS is its instance for trading strategies, which is why solving the inverse problem with language models and finding alpha are versions of one task. Second, the ceiling transfers with it. These systems recover equations expressible in a supplied set of operators, succeeding at the solution level of Table 5 by recombining primitives, and they tend to recover known laws; a governing equation outside the supplied primitives is not found. Progress on LLM-driven inverse-problem solving is therefore real and fast, but it is progress on search and certification within a supplied space, which is the capability LATSS validates and the boundary the benchmark below probes.

9.6 A benchmark for the jump

Of the directions in this section, the benchmark is the one we most want built. It is the single experiment that converts the question of machine invention from a debate into a measured quantity, and the only one whose outcome could overturn this paper’s central claim, so we set it out in detail. The requirement is a way to score the jump, and we propose the following protocol, of which Section 8.10 is a first component.

The protocol reuses the test-bed machinery of this paper, the synthetic panel, the dialed signal strength, and the same-class bar, but it inverts the question. In E1 the predictive feature is supplied to the searcher and the measured quantity is the power of the search; here the true driver is withheld from the vocabulary and the measured quantity is whether the agent asks for it. The scored action is not combining given features into a strategy but requesting, in natural language, a measurement that was never supplied.

1. **World.** Construct a synthetic panel whose true driver V is a *nameable* quantity that is absent from the supplied vocabulary, for example a calendar or turn-of-month seasonal, an exogenous named series, or a regime-conditional interaction of two features. Signal strength is tunable as in Eq. (2), so power can be mapped against it.
2. **Vocabulary and affordance.** The agent receives a feature set that excludes V , together with an affordance to request a new measurement or data source described in natural language.
3. **Interface.** Each round the agent either proposes a strategy over current features (search) or proposes a new measurement by description (a candidate jump). A resolver instantiates a requested measurement when it matches an allowed library, in which V is instantiable once named.
4. **Resolver and metric.** An automated judge, with human adjudication, maps a natural-language request to whether it denotes V or a sufficient statistic of it. The primary metric is the *jump rate*, the fraction of worlds in which the agent names V without a hint, measured against a base rate estimated from a fixed menu of generic requests (momentum, volatility, volume, and the like).
5. **Positive control.** Human experts, or an oracle with domain access, attempt the same task; the jump must be achievable for them at a rate well above the base rate, or the world is ill-posed and is discarded.

6. **Certification.** A named-and-instantiated V must still yield a strategy that clears the same-class null-max bar (Section 5.1), so a jump counts only when it is both proposed and certified.

As a worked example, let V be a day-of-week or turn-of-month seasonal while the supplied features are price-only technicals with no calendar input. The agent must conjecture that returns depend on the calendar and request a calendar feature; the base-rate menu never includes it. Mechanical searchers cannot propose at all and score zero by construction, which is the point: this experiment is about the conjecture step, not the search step. Related efforts give templates for the machinery, including benchmarks that control an agent’s prior knowledge in interactive discovery [41] and agentic loops that automate sequential falsification of proposed hypotheses [42].

Criteria for progress. A system moves the needle when its jump rate is significantly above the base rate, with provenance showing V was neither hinted nor inferable from the supplied features, certified by the bar, across many worlds and seeds. Such a result would be the first evidence against the ceiling of Section 8, and it is exactly the falsification condition stated there.

The design space. The benchmark is a family of tasks, not a single one, graded by how far the true driver sits from the supplied vocabulary. The easiest worlds hide a driver one short description away, such as a calendar effect absent from price-only features; harder worlds require an exogenous series the agent must think to pull in, or a regime-conditional interaction that only a structural hypothesis would suggest; the hardest place the driver in a relation no generic menu covers. Three arms make the comparison informative: a mechanical searcher that cannot propose and sets the floor at zero, a base-rate arm that proposes from a fixed generic menu, and the system under test. The human positive control sets the ceiling and certifies that each world is solvable, so the jump rate is read between a floor that is zero by construction and a ceiling a person can reach.

Role of the benchmark. The test is deliberately not about trading. Its subject is whether an agent can conjecture a variable outside the space it was given, with alpha as one instance, so a benchmark that works here transfers to scientific discovery more broadly, and the controlled-prior and sequential-falsification systems already in the literature [41, 42] supply much of the machinery. It also inherits the discipline of this paper: a proposal counts only when it is named, with provenance ruling out retrieval, and certified by a same-class bar, so the benchmark cannot be gamed by fluent but unfalsifiable suggestions. For these reasons we regard it as the most valuable single piece of future work the paper points to, and the natural next build on top of the certification layer established here.

9.7 Strengthening the measurement tool

These do not discover alpha, but they are prerequisites for trusting a future claim that something did. First, richer nulls. The sign-flip null of Section 5.2 assumes directional signal; a by-day sign flip, a cross-sectional permutation of returns across names, and block or stationary bootstraps would establish the bar’s operating characteristics under realistic temporal and cross-sectional dependence. Second, open-ended capacity. The capacity sweep (Section 7.5) runs on a finite random-tree class; extending it toward open-ended code-writing search would test whether the bar’s power collapses in the regime the VC scale predicts but we do not measure. Third, a fuller LLM power curve on real, nameable feature sets, where a model’s semantic prior might beat the blind mechanical search that matched it on opaque synthetic features (Section 7.6).

9.8 Enlarging the hypothesis space

The realistic near-term source of additional alpha is a wider vocabulary, searched and certified with the same discipline. A model can propose new measurements and data sources, filing text, supplier networks, options-surface features, and other alternative data, which are then constructed and certified against the null-max bar. This widens the supplied space rather than escaping it, but it is where practical gains are most likely, and the certification layer holds the additions to the same bar. A complementary direction penalizes proposals that are near-duplicates of known factors, by abstract-syntax-tree or embedding distance to a crowded-factor corpus, steering the search toward less-crowded regions and the durability that economic novelty requires.

9.9 Directions and milestones

Each remaining direction attaches to a measurable milestone on that benchmark.

Provenance. Ruling out retrieval is a precondition for any jump claim. A training-cutoff-controlled protocol, pre-registration of the vocabulary, and corpus near-duplicate checks on proposals separate invention from recall. Milestone: a model with no exposure to V 's domain still names it above the base rate.

Anomaly-triggered conjecture. Discovery starts from a misfit. A loop that surfaces residual predictability the vocabulary cannot explain and then aims conjecture at those residuals is the operational form of the trigger, a capacity current models lack unprompted [6]. Milestone: a measured rise in jump rate when the trigger is enabled.

Grounded experimentation. Grounding can be supplied externally (Section 8.7). A simulator the agent can intervene in, running counterfactuals and synthetic regimes, lets it separate a mechanism from a correlation, the closest handle on the manipulative-abduction step argued to be missing [2]. Milestone: proposed drivers that survive out of sample more often with grounding than without.

Mechanism-first architecture. A staged loop proposes a mechanism in natural language, derives its implications, specifies the feature that would detect it, builds the data, and certifies the result. Milestone: a certified driver that is also corpus-novel under the provenance checks above.

Across all of these the certification layer of this paper is a prerequisite. A jump is credible only when its champion clears a correctly matched bar and its provenance is established, so a trustworthy test of invention is built on a trustworthy test of skill.

10 Limitations

The validation uses one cross-validation scheme and the null-max bar as the gate, on a synthetic cross-sectional oracle, which is a clean test of recovery rather than a model of how real alpha is distributed. Certification is in sample against the bar by design, so the guarantee concerns the strategy class's capacity to fit noise, not out-of-sample performance. The capacity result (E4) is tested to ten thousand candidates and holds in part by construction; it does not rule out a loss of power for a fixed weak signal at larger budgets, and the sign-flip null tests directional signal, valid for this long-only task but not necessarily for a class with a non-directional overfitting channel. The LLM comparison (E7) uses 3 seeds at three signal levels rather than a full LLM power curve, and the search driver's per-run bar differs from the fixed mechanical bar, so we certify against the fixed

bar for comparability. The ceiling argument is reasoned rather than proved: a universal negative is not provable, and the measurement in Section 8.10 establishes the vocabulary boundary rather than an LLM-specific failure to jump.

11 Conclusion

We presented LATSS and validated the loop as a measurement tool rather than claiming a trading strategy. On synthetic controls it rejects pure noise, with the certification frequency on noise holding at 0 of 30 as the search budget grows to ten thousand candidates, recovers and certifies injected signal from an active Sharpe near 0.66, and loses validity only under a mis-specified bar that certifies noise in two of every three runs (20 of 30). In a three-seed comparison the LLM-driven search recovers signal at the same threshold as mechanical search, with no evidence of an advantage over the same vocabulary in this experiment. We then examined the ceiling. LATSS, like other LLM alpha-mining systems, searches a human-supplied hypothesis space rather than performing the abductive step that creates a new one, and withholding the predictive feature drives recovery to zero at every signal strength. External scaffolding can supply the grounded world model the LLM lacks; it cannot supply the step that creates a new hypothesis space. Our experiments support treating an LLM-driven loop as a measurement tool for signal represented in its supplied vocabulary, rather than as evidence that the loop originates new economic hypotheses.

Read as an inverse problem, LATSS is Tarantola’s procedure made concrete: a learned prior proposes models, the backtest solves the forward problem, and the null-max bar falsifies, so certification means survival against a matched null rather than a best fit. The limit is then categorical. A stateless inference engine samples within its distribution, and invention is a step outside it, so the route to a system that discovers new alpha is not a larger model but an agent that supplies what the model cannot: a way to enlarge the hypothesis space, a proposal heuristic aimed at its uncrowded regions, and a benchmark that can tell a genuine jump from retrieval. The certification layer validated here is the prerequisite for any such claim.

Reproducibility. The loop, the mechanical searchers, the skill bars, the cross-validation, the synthetic test-bed, and the scripts producing every table are open source at <https://github.com/vincent212/llm-assisted-trading-strategy-search>.

References

- [1] B. Romera-Paredes, M. Barekatain, A. Novikov, et al. Mathematical discoveries from program search with large language models (FunSearch). *Nature*, 2023.
- [2] T. Zahavy. LLMs Can’t Jump (position paper). DeepMind, 2026. <https://www.tomzahavy.com/projects/llms-cant-jump>.
- [3] C. Si, D. Yang, and T. Hashimoto. Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers. arXiv:2409.04109, 2024.
- [4] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292, 2024.
- [5] J. Beel, M.-Y. Kan, and M. Baumgart. Evaluating Sakana’s AI Scientist for Autonomous Research: Wishful Thinking or an Emerging Reality? arXiv:2502.14297, 2025.

- [6] A. W. Ding and S. Li. Generative AI lacks the human creativity to achieve scientific discovery from scratch. *Scientific Reports*, 2025. DOI 10.1038/s41598-025-93794-9.
- [7] R. Balestriero, J. Pesenti, and Y. LeCun. Learning in High Dimension Always Amounts to Extrapolation. arXiv:2110.09485, 2021.
- [8] Y. LeCun on world models. *MIT Technology Review*, 2026. <https://www.technologyreview.com/2026/01/22/1131661/yann-lecuns-new-venture-ami-labs/>.
- [9] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- [10] D. H. Bailey and M. López de Prado. The Deflated Sharpe Ratio. *Journal of Portfolio Management*, 2014.
- [11] V. Maciejewski. What Is the Null-Max Bar, and Could It Replace the Deflated Sharpe Ratio? 2026. <https://vincentmayeski.substack.com/p/what-is-the-null-max-bar-and-could>.
- [12] V. N. Vapnik. *Statistical Learning Theory*. Wiley, 1998.
- [13] P. L. Bartlett and S. Mendelson. Rademacher and Gaussian Complexities. *JMLR*, 3:463–482, 2002.
- [14] AlphaAgent: LLM-Driven Alpha Mining with Regularized Exploration to Counteract Alpha Decay. arXiv:2502.16789, 2025.
- [15] Z. Kou, H. Yu, J. Luo, et al. Automate Strategy Finding with LLM in Quant Investment. arXiv:2409.06289, 2024 (Findings of EMNLP 2025).
- [16] F. Liu et al. Cognitive Alpha Mining via LLM-Driven Code-Based Evolution. arXiv:2511.18850, 2025.
- [17] Celestial Quant Lab. Hubble: An LLM-Driven Agentic Framework for Safe, Diverse, and Reproducible Alpha Factor Discovery. arXiv:2604.09601, 2026.
- [18] J. Han et al. QuantaAlpha: An Evolutionary Framework for LLM-Driven Alpha Mining. arXiv:2602.07085, 2026.
- [19] Z. Weng, S. Zhang, T. Wang, and Y. Xia. AlphaLogics: A Market Logic-Driven Multi-Agent System for Scalable and Interpretable Alpha Factor Generation. arXiv:2603.20247, 2026.
- [20] H. Luo, Y. Zhang, and C. Liu. EFS: Evolutionary Factor Searching for Sparse Portfolio Optimization Using Large Language Models. arXiv:2507.17211, 2025.
- [21] M. R. Islam. The Evolution of Alpha in Finance: Harnessing Human Insight and LLM Agents. arXiv:2505.14727, 2025.
- [22] A. Novikov et al. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv:2506.13131, 2025.
- [23] MadEvolve: Evolutionary Optimization of Trading Systems with Large Language Models. arXiv:2605.23007, 2026.
- [24] QuantEvolve: an evolutionary quantitative trading strategy system (open source), 2025. <https://github.com/tarsyang/quantevolve>.

- [25] Alpha-GPT: Human-AI Interactive Alpha Mining for Quantitative Investment. arXiv:2308.00016, 2023.
- [26] Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning. KDD, 2023.
- [27] C. Bergmeir, R. J. Hyndman, and B. Koo. A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.
- [28] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu. The Probability of Backtest Overfitting. *Journal of Computational Finance*, 2017.
- [29] C. S. Peirce. *Collected Papers of Charles Sanders Peirce*. Harvard Univ. Press, 1931–1958. (On abduction; see also the Stanford Encyclopedia of Philosophy entry, “Abduction”.)
- [30] K. R. Popper. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge, 1963. (See also *The Logic of Scientific Discovery*, 1959.)
- [31] T. S. Kuhn. *The Structure of Scientific Revolutions*. Univ. of Chicago Press, 1962.
- [32] I. Lakatos. *The Methodology of Scientific Research Programmes*. J. Worrall and G. Currie, eds. Cambridge Univ. Press, 1978.
- [33] N. R. Hanson. *Patterns of Discovery*. Cambridge Univ. Press, 1958.
- [34] G. Schurz. Patterns of Abduction. *Synthese*, 164(2):201–234, 2008.
- [35] P. Langley, H. A. Simon, G. L. Bradshaw, and J. M. Zytkow. *Scientific Discovery: Computational Explorations of the Creative Processes*. MIT Press, 1987.
- [36] A. Tarantola. Popper, Bayes and the inverse problem. *Nature Physics*, 2:492–494, 2006.
- [37] A. Tarantola. *Inverse Problem Theory and Methods for Model Parameter Estimation*. SIAM, 2005.
- [38] P. Shojaei, K. Meidani, S. Gupta, A. Barati Farimani, and C. K. Reddy. LLM-SR: Scientific Equation Discovery via Programming with Large Language Models. arXiv:2404.18400, 2024 (ICLR 2025).
- [39] P. Ma, T.-H. Wang, M. Guo, Z. Sun, J. B. Tenenbaum, D. Rus, C. Gan, and W. Matusik. LLM and Simulation as Bilevel Optimizers: A New Paradigm to Advance Physical Scientific Discovery. arXiv:2405.09783, 2024 (ICML 2024).
- [40] LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models. arXiv:2504.10415, 2025 (ICML 2025).
- [41] PhysGym: Benchmarking LLMs in Interactive Physics Discovery with Controlled Priors. arXiv:2507.15550, 2025.
- [42] Automated Hypothesis Validation with Agentic Sequential Falsifications. arXiv:2502.09858, 2025.